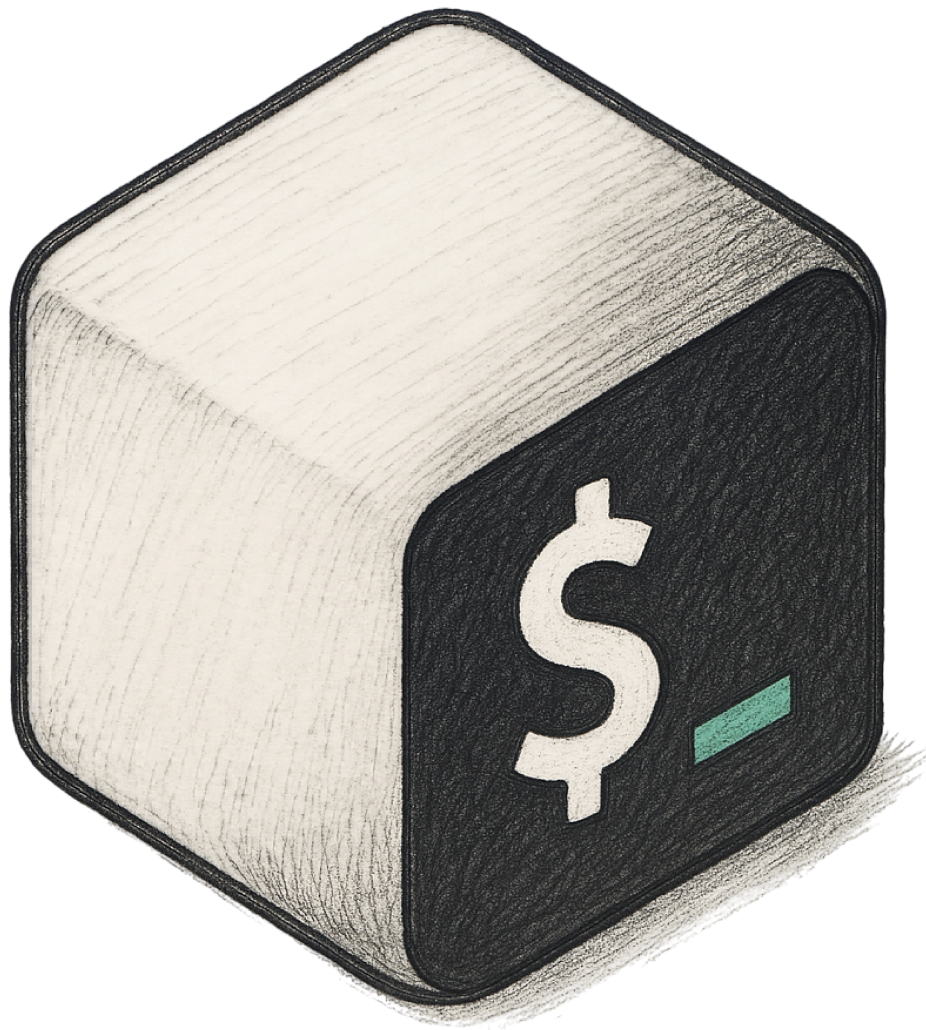


BASH SCRIPTING HANDBOOK



[SYSXPLORE.COM](https://sysxplore.com)

TRÄW

Bash Scripting Handbook

sysxplore.com

Version v1.0, 01.05.2025

Table of Contents

Preface	1
How to Use This Book	1
Conventions Used in This Book	1
Contact	2
Foreword	3
1. Getting started	4
1.1. Variables	4
1.2. Comments	4
1.3. Special Parameters	5
1.4. Functions	6
1.5. Conditions	6
1.6. Command Substitution	7
1.7. Brace Expansion	7
1.8. Operators in Bash	8
1.8.1. Arithmetic Operators	8
1.8.2. Assignment Operators	9
1.8.3. Relational Operators	9
1.8.4. Logical Operators	10
1.8.5. Bitwise Operators	10
1.8.6. Ternary Operator	10
2. Bash Built-in Commands	12
2.1. Built-ins	12
2.2. Built-ins: C–D	12
2.3. Built-ins: E–F	13
2.4. Built-ins: G–P	13
2.5. Built-ins: R–Z and Symbols	14
3. Bash Arrays	16
3.1. Array Declaration Syntax	16
3.2. Storing Values in Arrays	16
3.3. Getting Values from Arrays	17
3.4. Indexes and Metadata	18
3.5. Destroying Arrays	19
3.6. Array Discussion	19
3.6.1. Compound Indexed Array Assignment	19
3.6.2. Indexed Array Assignment	20
3.6.3. Indexed Array Assignment Using Brace Expansion	20
3.6.4. Key-Based Associative Array Assignment	20
3.6.5. Compound Associative Array Assignment	20
3.6.6. Working with Associative Arrays	21
3.6.7. Appending Values to Arrays	21
3.6.8. Iterating Over an Indexed Array	22

3.6.9. Iterating Over an Associative Array	22
3.6.10. Merging / Concatenating Arrays	22
3.6.11. Array Arguments to Functions	23
4. Bash Conditions and Flow Control	24
4.1. Numeric Conditions	24
4.2. String Conditions	25
4.3. File Conditions	25
4.4. Compound Conditions	26
4.5. More Conditions	27
4.6. if Statements	27
4.6.1. Basic Syntax	27
4.6.2. if...else	28
4.6.3. if...elif...else	28
4.6.4. Using [[...]] vs [...] vs ...	28
4.6.5. if with Command Execution	29
4.6.6. Nested if Statements	29
4.6.7. Compound Conditions with && and	29
4.6.8. Grouping with Double Brackets	30
4.7. The test Command	30
4.7.1. Basic Syntax	30
4.7.2. Examples	30
4.7.3. Why Use test?	31
4.8. Advanced if-then Features	31
4.9. case Statement	31
4.10. Loops That Use Conditions	33
4.10.1. while	33
4.10.2. until	33
5. Bash Loops	34
5.1. Basic for Loop	34
5.2. C-like for Loop	34
5.3. Using Ranges	35
5.4. Bash while Loop	35
5.4.1. Counting Up	35
5.4.2. Counting Down	35
5.4.3. while true Loop	36
5.4.4. Reading Files with while	36
5.5. until Loop	37
5.6. continue Statement	37
5.7. break Statement	38
5.8. Additional Notes on Loops	38
5.8.1. Nested Loops	38
5.8.2. Looping Over Arrays with read -a	39
5.8.3. Looping Over Command Output Safely	39

5.8.4. Creating an Infinite Loop with a Delay	39
5.8.5. Performance and Portability Tip	40
5.9. The select Loop	40
5.9.1. Basic Syntax	40
5.9.2. Menu Example	41
5.9.3. Handling Invalid Input	41
5.9.4. Changing the Prompt	41
6. Bash Functions	43
6.1. Defining Functions	43
6.2. Returning Values	43
6.3. Raising Errors with return	43
6.4. Recursive Functions	44
6.5. Fork Bomb	44
6.6. More on Functions	45
6.6.1. Variables Are Global by Default	45
6.6.2. Unsetting or Overwriting Functions	45
6.6.3. Listing Functions	45
6.6.4. Valid Function Names	46
6.6.5. Exporting Functions to Subshells (Bash 4+)	46
7. Bash Arithmetic Operations	47
7.1. Bash Arithmetic Operations	48
7.1.1. Using let to Perform Arithmetic Operations	48
7.1.2. Using expr to Perform Arithmetic Operations	49
7.1.3. Using ... for Arithmetic Operations	50
7.2. Floating Point Arithmetic with bc	51
7.2.1. Floating Point Arithmetic with awk	51
7.3. Arithmetic with Arrays	51
7.4. Preventing Division by Zero	52
8. Bash Parameter Expansion	53
8.1. Syntax	53
8.2. Substrings	53
8.3. Length	54
8.4. Default Values	54
8.5. Substitution	54
8.6. Slicing	54
8.7. Basepath & Dirpath	55
8.8. Transform	55
9. Command Line Chaining	56
9.1. & – Run in the Background	56
9.2. && – Run If Previous Succeeds	56
9.3. – Run If Previous Fails	56
9.4. ; – Always Run Next Command	56
9.5. ! – Negate Exit Status	57

9.6. && – Simulate If-Else	57
9.7. – Pipe Output to Another Command	57
9.8. >, >>, < – Redirect Input and Output	57
9.9. \ – Continue Command on New Line	58
9.10. () – Subshell Execution	58
9.11. { } – Grouping in Current Shell	58
10. Command Substitution	59
10.1. Syntax	59
10.2. Basic Example	59
10.3. Assigning Values Dynamically	59
10.4. Using Output as Arguments	59
11. Process Substitution	61
11.1. Syntax	61
11.2. Comparing Files	61
11.3. Comparing Command Output	61
11.4. Searching Through Output	61
11.5. Sorting and Saving Output	61
11.6. Opening Multiple Files	62
11.7. Transferring Compressed Files Over SSH	62
11.8. Benefits	62
12. Bitwise Operators in Bash	63
12.1. What Are Bits and Bytes?	63
12.2. Supported Bitwise Operators in Bash	63
12.3. Bitwise AND (&)	63
12.4. Bitwise OR ()	64
12.5. Bitwise XOR (^)	64
12.6. Bitwise NOT (~)	64
12.7. Bitwise Left Shift (<<)	64
12.8. Bitwise Right Shift (>>)	65
12.9. Practical Uses of Bitwise Operators	65
13. Quoting in Bash	66
13.1. Using Double Quotes	66
13.1.1. Expanding Variables	66
13.1.2. Command Substitution Inside Double Quotes	66
13.1.3. What Gets Interpreted Inside Double Quotes?	67
13.1.4. Breaking Long Lines	67
13.2. Using Single Quotes	67
13.2.1. Printing a Variable Name	67
13.2.2. Disabling Wildcards and Substitution	68
13.2.3. When to Use Single Quotes	68
13.3. Using Backslashes	68
13.3.1. Escaping Dollar Signs	68
13.3.2. Escaping Quotes	68

13.3.3. When to Use Backslashes	69
14. Globbing in Bash	70
14.1. What Is Globbing?	70
14.2. Matching Any String with *	70
14.3. Matching a Single Character with ?	71
14.4. Matching Specific Characters with []	71
14.5. Matching Hidden Files	72
15. Pipelines and Redirections	73
15.1. Pipelines in Bash	73
15.1.1. What Is a Pipeline?	73
15.1.2. Basic Example	73
15.1.3. Filtering and Transforming Data	73
15.1.4. Combining Pipelines with Redirection	74
15.1.5. A quick practical example	74
15.2. Redirections in Bash	75
15.2.1. Redirection Operators	75
16. Exit Status and the exit Command	79
16.1. What Is an Exit Status?	79
16.2. Exit Status with Pipelines	80
16.3. The exit Command	80
16.4. Using exit in Scripts	81
16.5. What If You Don't Pass an Argument?	81
17. Bash History	83
17.1. Viewing and Managing Command History	83
17.1.1. History Event Designators	83
17.2. Word Designators	84
17.3. Modifiers for Word Designators	84
17.3.1. History Modifiers	85
17.3.2. History Slices	86
18. Bash Options	87
18.1. Common Bash Options	87
18.2. Additional Bash Options	89
18.2.1. The shopt Builtin	89
18.2.2. Exit Code Behavior	89
18.2.3. shopt Options Summary	90
18.3. Additional Shell Options	90
19. Regular Expressions in Bash	92
19.1. Types of Regular Expressions	92
19.2. Anchors	92
19.3. Character Classes	92
19.4. Shorthand Character Classes	93
19.5. POSIX Character Classes (for BRE/ERE)	93
19.6. Quantifiers	95

19.7. Escaping Special Characters	95
19.8. Groups and Ranges	96
19.9. Pattern Modifiers	96
19.10. Assertions	97
19.11. String Replacement Variables	97
19.12. Escape Sequences	97
19.13. Special Characters	98
19.14. Search for Lines That Start with a Word	98
19.15. Match IP Addresses Using <code>grep</code>	99
19.16. Find Email Addresses in a Text File	99
19.17. Replace Tabs with Commas Using <code>sed</code>	99
19.18. Print Lines That Don't Contain a Pattern	99
19.19. Extract Usernames from <code>/etc/passwd</code>	100
Appendix A: Terminal Shortcuts Reference	101
A.1. Bash Navigation	101
A.2. Bash Control and Process Handling	101
A.3. Bash History Shortcuts	101
A.4. Bash History Expansions	102
A.5. Bash Line Editing	102
A.6. Bash Autocomplete and Information	103
A.7. Bash Special Parameters	103
A.7.1. Positional Parameters	103
A.7.2. Other Special Parameters	104
A.8. Bash Special Parameters	104
A.8.1. <code>\$?</code> — Exit Status of Last Command	104
A.8.2. <code>\$0</code> — Name of the Script	105
A.8.3. <code>\$#</code> — Number of Positional Parameters	105
A.8.4. <code>\$@</code> and <code>\$*</code> — All Positional Parameters	105
A.8.5. <code>\$!</code> — PID of Last Background Process	105
A.8.6. <code>\$\$</code> — PID of Current Shell	106
A.8.7. <code>\$-</code> — Current Shell Options	106
A.8.8. <code>\$?</code> , <code>\$#</code> , <code>\$@</code> , and more...	106
A.9. Bash Signals	106
A.9.1. Common Signals	106
A.9.2. Viewing Available Signals	107
A.9.3. Sending Signals	107
A.9.4. Handling Signals in Scripts	107
A.10. Prompt Customization Examples	108
A.10.1. Basic PS1 Example	108
A.10.2. Add Color to the Prompt	108
A.10.3. Customize PS2	108
A.10.4. Customize PS4	109
A.10.5. Prompt with Git Branch	109

A.11. Reserved Bash Keywords	109
A.12. Debugging Bash Scripts	111
A.12.1. Enabling Debug Mode with -x	111
A.12.2. Syntax Checking with -n	112
A.12.3. Verbose Mode with -v	112
A.12.4. Use trap to Catch Errors	112
A.12.5. Print Debug Messages Manually	112
A.12.6. Use return and exit Codes	113
A.12.7. Debugging Loops and Conditionals	113
A.12.8. Summary	113

Preface

This is a hands-on guide for anyone learning Bash scripting or working with the command line. It's written as a collection of short, example-focused notes. It's made for beginners who want to get started fast, and for experienced users who need a solid reference close at hand.

Whether you're writing scripts, automating tasks, fixing things, or just exploring what the shell can do, these notes are meant to be clear, direct, and ready to use. I started this collection to bring together the key Bash concepts, real-world examples, and small tricks I wish I had all in one place when I began.

How to Use This Book

You don't need to read everything in order. Skip around, copy examples, test commands, and come back whenever something doesn't quite make sense. This is your toolkit, use it how you like. I've structured it to start with the basics and gradually move into more advanced topics.

Conventions Used in This Book

Commands, filenames, variables, and syntax are shown in monospace like this: `echo $HOME`.

Commands meant to be run in the terminal appear like this:

```
$ echo "Hello, world"
```

The `$` represents the shell prompt—you don't need to type it.

Longer scripts or multi-line examples are also shown in code blocks:

```
#!/bin/bash
name="Alice"
echo "Welcome, $name!"
```

Throughout the book, you'll also see side notes:



Tip: Helpful shortcuts or small tricks to save time or avoid extra steps.



Note: Extra context, background, or alternative ways to think about a topic.



Important: Things you'll want to pay attention to, especially if a command behaves differently depending on the shell or system.



Caution: Used when something could have unintended consequences like commands that modify system files, affect user data, or involve elevated privileges.



Warning: Mistakes to avoid, risky commands, or cases where something might break if used carelessly.

All examples are written for Bash 5 and tested on modern Linux distributions. If you're using a different shell or operating system (like macOS or WSL), results may vary slightly.

Contact

If you have questions, suggestions, or spot something that could be improved in these notes or any mistakes, feel free to reach out.

You can message me on X: [@thatstraw](#) Or send an email to: book@sysxplore.com

More from the Author

If you found this book helpful, you might enjoy some of the other content and projects I'm working on:

- My beginner-friendly Linux book: firststepswithlinux.com
- Linux tutorials and deep dives: sysxplore.substack.com
- Visual cheatsheets and graphics: [Cheatsheets on Gumroad](#)
- My blog: sysxplore.com

Foreword

Bash scripting is more than just a technical skill. It's a way to automate tasks, solve problems, and get more out of your computer. The command line is still one of the most powerful tools for developers, sysadmins, and anyone who likes to dig deeper into how things work.

This book is a collection of notes, examples, and tips pulled from real situations. It's meant to help you get comfortable with Bash, whether you're just starting out or looking to sharpen what you already know. Everything here is written to be practical and easy to follow.

A Note to the Reader

Try things out. Break stuff. Change commands and see what happens. That's how you'll really learn Bash.

These notes aren't meant to be read once and forgotten, they're here to come back to when you're stuck or curious. Use them as a guide while you experiment and build your own scripts.

Chapter 1. Getting started

In this section, you'll learn the foundational elements of Bash scripting, including how to define and use variables, write comments, and create simple scripts. Each concept builds on the last, helping you get comfortable writing your own Bash scripts from scratch.

1.1. Variables

Variables in Bash are used to store and reference data. You assign a value using the = operator, and retrieve the value using the \$ symbol.

```
NAME="TRAW"
```

```
echo ${NAME}      # => TRAW :This prints the variable value
echo $NAME        # => TRAW :This prints the variable value
echo "$NAME"      # => TRAW :This prints the variable value
echo '$NAME'      # => $NAME : This will print the exact string, not the value
echo "${NAME}!"   # => TRAW! : Concatenation of variable value with !
```

Important

When declaring variables, there must be no space between the variable name, the = sign, or the value:

```
NAME = "TRAW"    # => Error (due to spaces)
```

To **assign** a value, you don't use the dollar sign (\$). But when you **reference** a variable's value, you do:

- Assignment: NAME="TRAW"
- Reference: echo \$NAME

1.2. Comments

Comments help explain your code and are ignored during execution. Use the # symbol for single-line comments.

```
# This is an inline Bash comment.
```

For multi-line comments, Bash doesn't have a dedicated syntax like other languages, but you can use a : ' block trick.

```
: '  
This is a  
Multi-line comment  
in bash  
  
If it looks very neat  
'
```



For multi-line comments, use : ' to open and ' to close the block.

Here's a simple Bash script that stores a greeting and prints it to the terminal.

```
#!/usr/bin/env bash  
  
greet="world World"  
echo "$greet!, I'm new here."  
# => Hello world! I'm new here.
```

You can save this script to a file and run it in two ways:

```
$ chmod +x helloworld.sh  
$ ./helloworld.sh  
  
# or just  
$ bash helloworld.sh
```

1.3. Special Parameters

Special parameters in Bash are predefined variables that give you information about arguments, the shell environment, and the script's execution state.

Parameter	Description
\$1...\$9	Parameters 1 to 9 (positional arguments)
\$0	Name of the script or the shell
\$1	First argument passed to the script or function
\${10}	Positional parameter 10 (and beyond)
\$#	Total number of positional parameters
\$*	All positional parameters as a single word

Parameter	Description
\$@	All positional parameters as separate words
\$\$	Process ID of the current shell
\$-	Current shell options
\$_	Last argument of the previous command
!	PID of the last background command
\$?	Exit status of the last foreground command



\$* and \$@ both expand all positional parameters, but behave differently when quoted. We'll explore that further in later sections.

For deeper reference, see: [Bash Hackers Wiki](#)

1.4. Functions

Functions in Bash allow you to group reusable code into named blocks. You can define them with or without the function keyword.

```
#!/usr/bin/env bash

get_username() {
    echo "TRAW"
}

# Or using the function keyword
function get_username() {
    echo "TRAW"
}

echo "You are $(get_username)"
```



Functions can return values using `echo`, and the result can be captured via command substitution like `$(...)`.

1.5. Conditions

Conditional statements in Bash allow you to make decisions in your scripts. The `if` statement evaluates whether a condition is true, and executes the appropriate block of code.

```
#!/usr/bin/env bash

string="test"

if [[ -z "$string" ]]; then
    echo "String is empty"
elif [[ -n "$string" ]]; then
    echo "String is not empty"
fi
```



- -z checks if a string is empty.
- -n checks if a string is not empty.
- Use `[[ ... ]]` for more robust conditional expressions.

1.6. Command Substitution

Command substitution lets you capture the output of a command and use it as part of another command or assignment.

```
# => I'm TRAW
echo "I'm $(whoami)"

# Similar to:
echo "I'm `whoami`"
```



The modern syntax `$(command)` is preferred over backticks `command` because it's easier to read and allows nesting.

1.7. Brace Expansion

Brace expansion is a convenient way to generate strings or sequences in Bash.

Example	Description
<code>\$ echo {A,B}.txt</code>	Outputs: A.txt B.txt
<code>{A,B}</code>	Expands to: A B
<code>{1..5}</code>	Expands to: 1 2 3 4 5



Brace expansion is useful for generating multiple similar strings without writing them out manually. Great for looping over files or ranges.

For more examples, see: [Bash Hackers Wiki](#)

1.8. Operators in Bash

Operators allow you to perform operations on values and variables in Bash. Bash provides various types of operators for different purposes:

- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Ternary Operator

This article will provide a detailed explanation of each type of operator in Bash scripting.

1.8.1. Arithmetic Operators

Arithmetic operators allow you to perform mathematical operations on integers. They combine integer variables and values to produce a new integer result.

Below is a list of all the arithmetic operators available in Bash:

Operator	Description	Example
+	Addition	<code>\$a + \$b</code>
-	Subtraction	<code>\$a - \$b</code>
*	Multiplication	<code>\$a * \$b</code>
/	Division	<code>\$a / \$b</code>
%	Modulus (remainder)	<code>\$a % \$b</code>
**	Exponentiation (a to the power b)	<code>\$a ** \$b</code>

The addition, subtraction, multiplication, and division operators work as expected for mathematical operations.

The modulus (%) operator returns the remainder of the division between two integers. This is useful for checking if a number is even or odd, among other uses.

1.8.2. Assignment Operators

Assignment operators store values or the result of an expression into a variable.

Operator	Description	Example
=	Simple assignment	a=5
+=	Add and assign	a+=2
-=	Subtract and assign	a-=3
=	Multiply and assign	a=4
/=	Divide and assign	a/=5
%=	Modulus and assign	a%=6

Simple assignment allows storing values in a variable. The combined assignment operators like `=` and `-=` allow modifying variables more concisely. For instance, `a=b` is the same as `a = a + b`.

1.8.3. Relational Operators

Relational operators are used for comparing integers and strings to evaluate conditions.

Operator	Description	Example
-eq	Equal to	[\$a -eq \$b]
-ne	Not equal to	[\$a -ne \$b]
-gt	Greater than	[\$a -gt \$b]
-ge	Greater than or equal to	[\$a -ge \$b]
-lt	Less than	[\$a -lt \$b]
-le	Less than or equal to	[\$a -le \$b]
<	Less than (strings)	["\$a" < "\$b"]
<=	Less than or equal (strings)	["\$a" <= "\$b"]
>	Greater than (strings)	["\$a" > "\$b"]
>=	Greater than or equal (strings)	["\$a" >= "\$b"]
==	Equal to	[\$a == \$b]
!=	Not equal to	[\$a != \$b]

`-eq`, `-ne`, `-gt`, `-ge`, `-lt`, and `-le` work on integers, while `<`, `=`, `>`, and `>=` work on string sorting order. These are commonly used in `if` statements and loops to control program flow. The equal (`==`) and not equal (`!=`) operators are useful for comparing integers.

1.8.4. Logical Operators

Logical operators are used for combining and negating conditional expressions.

Operator	Description	Example
<code>!</code>	Negation	<code>! [\$a -eq \$b]</code>
<code>&&</code>	AND	<code>[\$a -eq 5] && [\$b -eq 10]</code>
<code> </code>	OR	<code>[\$a -eq 5] [\$b -eq 10]</code>

The NOT operator (`!`) inverts a true condition to false or a false condition to true. The AND operator (`&&`) evaluates to true if both operands are true, while the OR operator (`||`) evaluates to true if either operand is true. These allow creating complex conditional logic in scripts.

1.8.5. Bitwise Operators

Bitwise operators manipulate integers at the bit level.

Operator	Description	Example
<code>&</code>	Bitwise AND	<code>\$a & \$b</code>
<code> </code>	Bitwise OR	<code>\$a \$b</code>
<code>~</code>	Bitwise NOT	<code>~\$a</code>
<code>^</code>	Bitwise XOR	<code>\$a ^ \$b</code>
<code><<</code>	Left shift	<code>\$a << 2</code>
<code>>></code>	Right shift	<code>\$a >> 2</code>

Bitwise operators treat integers as binary strings and set, unset, or toggle bits at specific positions. This allows bitmasking, toggling, and shifting values for flags and low-level binary operations.

1.8.6. Ternary Operator

The ternary operator allows simple conditional expressions.

Syntax	Description
<code><condition> ? <if true> : <if false></code>	Evaluates test condition and returns the "if true" or "if false" result

The ternary operator is structured as `condition ? resultIfTrue : resultIfFalse`. It tests the given condition and returns the specified result depending on whether the condition evaluated to true or false. This provides a concise way to assign values based on conditions.

Chapter 2. Bash Built-in Commands

Built-in commands are part of the shell itself and do not require launching a separate executable. Because they're compiled directly into the shell, they are generally faster and more efficient than external commands.

2.1. Built-ins

Command	Description
<code>.</code> (dot)	Same as <code>source</code> . Executes a script in the current shell environment.
<code>&</code>	Runs a job in the background.
<code>x</code>	Evaluates the arithmetic expression <code>x</code> .
<code>:</code>	No-op command. Does nothing and always succeeds (similar to <code>true</code>).
<code>[t]</code>	Tests the expression <code>t</code> (same as the <code>test</code> command).
<code>[[e]]</code>	Evaluates the conditional expression <code>e</code> . Preferred for complex conditions.
<code>alias</code>	Creates a shortcut or alias for another command.
<code>bg</code>	Resumes a suspended job in the background.
<code>bind</code>	Binds keyboard sequences to readline functions or macros.
<code>break</code>	Exits a <code>for</code> , <code>while</code> , <code>select</code> , or <code>until</code> loop early.
<code>builtin</code>	Forces execution of a shell built-in, bypassing any function or alias with the same name.

2.2. Built-ins: C–D

Command	Description
<code>caller</code>	Displays the call stack — useful inside functions and scripts.
<code>case</code>	Executes commands based on pattern matching.
<code>cd</code>	Changes the current working directory.
<code>command</code>	Runs a command while bypassing shell function or alias lookup.
<code>compgen</code>	Displays possible autocompletion options for a given word.
<code>complete</code>	Specifies how arguments should be completed for a command.
<code>compt</code>	Sets completion behavior for arguments.

Command	Description
<code>continue</code>	Skips the rest of a loop and begins the next iteration.
<code>coproc</code>	Starts a background process connected to the script via file descriptors.
<code>declare</code>	Declares variables and assigns attributes like <code>-a</code> for arrays or <code>-i</code> for integers.
<code>dirs</code>	Shows a list of remembered directories in the directory stack.
<code>disown</code>	Removes a job from the shell's job table (won't be affected by HUP signals).

2.3. Built-ins: E–F

Command	Description
<code>echo</code>	Prints text or variables to standard output.
<code>enable</code>	Enables or disables built-in shell commands.
<code>eval</code>	Combines arguments into a single string and evaluates them as a command.
<code>exec</code>	Replaces the current shell process with a different program.
<code>exit</code>	Exits the shell with a status code.
<code>export</code>	Makes variables available to child processes.
<code>false</code>	Always returns a non-zero exit status (indicating failure).
<code>fc</code>	Lists or edits commands from history.
<code>fg</code>	Resumes a background job in the foreground.
<code>for</code>	Iterates over a list of items, executing commands for each.
<code>function</code>	Declares a shell function.

2.4. Built-ins: G–P

Command	Description
<code>getopts</code>	Parses positional parameters for flags (used in argument parsing).
<code>hash</code>	Caches the location of commands to speed up lookup.

Command	Description
help	Displays help information for Bash built-in commands.
history	Shows the command history.
if	Runs a block of commands conditionally.
jobs	Lists current background and suspended jobs.
kill	Sends a signal (e.g., SIGKILL, SIGTERM) to a process by PID.
let	Performs arithmetic evaluation on arguments.
local	Declares a variable with local scope inside functions.
logout	Exits a login shell session.
mapfile	Reads lines from standard input into an array.
popd	Removes the top directory from the directory stack.
printf	Prints formatted output (more powerful than echo).
pushd	Adds a directory to the directory stack.
pwd	Displays the current working directory.

2.5. Built-ins: R–Z and Symbols

Command	Description
read	Reads one line of input and assigns it to variables.
readarray	Reads lines into an array (same as mapfile).
readonly	Marks a variable as read-only (cannot be modified).
return	Exits a function and optionally returns a value.
select	Presents a numbered menu to the user for input.
set	Sets shell flags or environment variables.
shift	Shifts positional parameters left by one.
shopt	Toggles optional shell behavior (like globstar).
source	Executes commands from a file in the current shell.
suspend	Suspends the shell until it receives SIGCONT.

Command	Description
<code>test</code>	Evaluates a condition and returns success or failure.
<code>time</code>	Measures the time taken by a command to execute.
<code>times</code>	Displays shell CPU time usage.
<code>trap</code>	Specifies commands to run on receiving a signal.
<code>true</code>	Always returns a successful (0) exit status.
<code>type</code>	Tells how a command would be interpreted (alias, function, builtin, etc.).
<code>typeset</code>	Declares variables (same as <code>declare</code> , legacy use).
<code>ulimit</code>	Sets resource usage limits.
<code>umask</code>	Sets default permission bits for new files/directories.
<code>unalias</code>	Removes one or more aliases.
<code>unset</code>	Unsets variables or shell functions.
<code>until</code>	Repeats a command block until a condition becomes true.
<code>wait</code>	Waits for background jobs to finish.
<code>while</code>	Repeats a command block while a condition remains true.
<code>{ c; }</code>	Groups commands in the current shell context.

Use the `help` command to see built-in command details right from your terminal:

Or to get help for a specific built-in:

```
help builtin_name
```

Chapter 3. Bash Arrays

Arrays let you store multiple values in a single variable. Bash supports both indexed and associative arrays. Here's how to declare them:

3.1. Array Declaration Syntax

Syntax	Description
<code>ARRAY=()</code>	Declares an indexed array and initializes it as empty. Also used to reset an existing array.
<code>ARRAY[0]=value</code>	Sets the first element of an indexed array. If the array doesn't exist yet, it's created.
<code>declare -a ARRAY</code>	Declares an indexed array without initializing it.
<code>declare -A ARRAY</code>	Declares an associative array. This is the only way to create associative arrays in Bash.

3.2. Storing Values in Arrays

There are many ways to assign values to arrays in Bash. This includes assigning individual elements, using compound assignment, copying from other arrays, and even reading from files.

Syntax	Description
<code>ARRAY[N]=VALUE</code>	Sets element N of an indexed array to VALUE. N can be any valid arithmetic expression .
<code>ARRAY[STRING]=VALUE</code>	Sets element STRING of an associative array to VALUE.
<code>ARRAY=VALUE</code>	Assigns VALUE to index 0 by default. This also works for associative arrays — VALUE is stored under key "0".
<code>ARRAY=(E1 E2 ...)</code>	Compound assignment for indexed arrays. Clears the array and assigns values starting at index 0. Does not work for associative arrays.
<code>ARRAY=([X]=E1 [Y]=E2 ...)</code>	Indexed array assignment using index-value pairs. X and Y are arithmetic expressions.
<code>ARRAY=([S1]=E1 [S2]=E2 ...)</code>	Compound assignment for associative arrays using string keys.
<code>ARRAY+=(E1 E2 ...)</code>	Appends elements to the existing array.

Syntax	Description
<code>ARRAY=("\${ARRAY[@]}" E1 ...)</code>	Alternative method for appending elements using array expansion.
<code>ARRAY=("\${ANOTHER_ARRAY[@]}")</code>	Copies elements from <code>ANOTHER_ARRAY</code> into <code>ARRAY</code> . Quotes are optional.
<code>ARRAY=("\${ARRAY[@]}" "\${ANOTHER_ARRAY[@]}")</code>	Concatenates two arrays. You can omit the quotes or use <code>*</code> instead of <code>@</code> .
<code>ARRAY=(\$(cat file)</code>	Reads a file and stores each word or line (depending on IFS) into an array.
<code>ARRAY=({N..M})</code>	Uses brace expansion to populate the array. <code>N</code> and <code>M</code> can be numbers or letters.



Compound assignment is very powerful and can be used to build arrays efficiently. Just be cautious: reassigning an array this way will unset previous values unless `+=` is used.

For clarification: When you use the subscripts `@` or `*` for mass-expanding arrays, their behavior matches that of `$@` and `$*` when [expanding positional parameters](#). It's worth reading that article to fully understand the nuances.

3.3. Getting Values from Arrays

You can access specific elements, the entire array, or a slice of elements using parameter expansion.

Syntax	Description
<code>\${ARRAY[N]}</code>	Gets the value at index <code>N</code> in an indexed array. If <code>N</code> is negative, it's counted backward from the highest assigned index (only for reading, not assigning).
<code>\${ARRAY[S]}</code>	Gets the value at key <code>S</code> in an associative array.
<code>\${ARRAY[@]}</code>	Expands to all elements (unquoted: expands to a word list).
<code>"\${ARRAY[@]}"</code>	Expands to all elements, each quoted separately.
<code>\${ARRAY[*]}</code>	Expands to all elements as a single word (if unquoted).
<code>"\${ARRAY[*]}"</code>	Expands to all elements as one quoted string.

Syntax	Description
<code>\${ARRAY[@]:N:M}</code>	Expands to a slice of M elements starting at index N (unquoted).
<code>"\${ARRAY[@]:N:M}"</code>	Quoted slice expansion — each element quoted separately.
<code>\${ARRAY[*]:N:M}</code>	Slice expansion with * (unquoted).
<code>"\${ARRAY[*]:N:M}"</code>	Slice expansion with *, quoted as a single string.



For clarification: When using `@` or `*` to expand arrays, their behavior mirrors how `$@` and `$*` behave with positional parameters.

- Quoted `"$@"` → each element separately quoted.
- Quoted `"$*"` → all elements quoted as one string.
- Unquoted `@` and `*` behave the same — they expand to a space-separated list of elements.

When using slicing, the same quoting rules apply. If you're getting unexpected results, check your Bash version and test in your shell.

3.4. Indexes and Metadata

You can inspect arrays using special parameter expansion patterns that give you access to their structure and metadata.

Syntax	Description
<code>\${ARRAY[0]}</code>	Gets the first element. See the Getting Values section.
<code>\${ARRAY[-1]}</code>	Gets the last element (backward index — read-only).
<code>\${ARRAY[*]}</code>	Expands to all elements. See the Getting Values section.
<code>\${ARRAY[@]}</code>	Same as above. See the Getting Values section.
<code>\${#ARRAY[@]}</code>	Expands to the number of elements in the array.
<code>\${#ARRAY[*]}</code>	Same as above — total number of items.
<code>\${#ARRAY}</code>	Expands to the length of the first element only.
<code>\${#ARRAY[N]}</code>	Expands to the length of the string at index N.
<code>\${#ARRAY[STRING]}</code>	Expands to the length of the associative array value at key STRING.
<code>\${#ARRAY[@]:N:M}</code>	Expands to the number of elements from index N up to N+M. M is optional.
<code>\${#ARRAY[*]:N:M}</code>	Same as above with * instead of @.

Syntax	Description
<code>\${!ARRAY[@]}</code>	Expands to a list of all indexes in the array.
<code>\${!ARRAY[*]}</code>	Same as above.

3.5. Destroying Arrays

You can remove individual elements or entire arrays using the `unset` built-in.

Syntax	Description
<code>unset -v ARRAY</code>	Unsets the entire array.
<code>unset -v ARRAY[@]</code>	Same — clears the entire array.
<code>unset -v ARRAY[*]</code>	Same — clears the entire array.
<code>unset -v 'ARRAY[N]'</code>	Unsets the element at index N. Always quote the index.
<code>unset -v ARRAY[STRING]</code>	Removes an element from an associative array at key STRING.
<code>ARRAY=(\${ARRAY[@]/STR*/})</code>	Removes elements matching a pattern (STR).
<code>ARRAY=(\${ARRAY[]/STR/})</code>	Same as above.



It's best to explicitly use `-v` with `unset` for clarity and portability.

See: [Bash Hackers Wiki](#)

3.6. Array Discussion

This section takes a deeper look at how arrays work in Bash. You'll explore different ways to assign values to both indexed and associative arrays, how to append and merge them, and how to loop through elements. We'll also cover how to pass arrays to functions, extract values, and remove elements using built-in tools and expansion syntax.

3.6.1. Compound Indexed Array Assignment

You can assign an entire array at once using parentheses. This sets all elements in one step, starting from index 0.

```
# Compound array assignment - sets the whole array
```

```
# NAMES => Creg Jan Anna
NAMES=('Creg' 'Jan' 'Anna')

# Using declare with compound assignment
declare -a Numbers=(1 2 3)
```

3.6.2. Indexed Array Assignment

You can also assign elements one at a time by specifying the index. The index can be any valid arithmetic expression.

```
NAMES[0]='Creg'
NAMES[1]='Jan'
NAMES[2]='Anna'
# NAMES => Creg Jan Anna
```

3.6.3. Indexed Array Assignment Using Brace Expansion

Brace expansion is a quick way to generate predictable patterns for array elements.

```
ARRAY1=(foo{1..2})      # => foo1 foo2
ARRAY2=({A..D})         # => A B C D
ARRAY3=({1..5})         # => 1 2 3 4 5
ARRAY4=({A..B}{1..2})   # => A1 A2 B1 B2
```

3.6.4. Key-Based Associative Array Assignment

Associative arrays store values using string keys. You must use `declare -A` to define one.

```
declare -A person

person[name]="Jay"
person[age]=22
person[eye_color]="blue"

# person => Jay 22 blue
```

3.6.5. Compound Associative Array Assignment

You can initialize all keys and values at once using a compound assignment.

```

: '
Individual mass-setting for associative arrays.
The named indexes (e.g., name, age) are strings.
'

declare -A person # Required declaration
person=(["name"]="Jay" [age]=22 ["eye_color"]="blue")

# person => Jay 22 blue

```

3.6.6. Working with Associative Arrays

Once an associative array is created, you can interact with it using standard Bash expansions.

```

declare -A person
person=(["name"]="Jay" [age]=22 ["eye_color"]="blue")

echo ${person[name]} # Get person's name
echo ${person[@]}    # Get all values
echo ${person[*]}    # Get all values
echo ${!person[@]}   # Get all keys
echo ${#person[@]}   # Get number of elements

unset -v person[age] # Delete age entry

```



All of these operations work the same way with indexed arrays — just use numeric indexes instead of string keys.

3.6.7. Appending Values to Arrays

You can add new elements to an array using the += operator or by expanding the current array into a new list.

```

NAMES=('Creg' 'Jan' 'Anna')

NAMES+=('Jay' 'Jimmy' 'Tom')
# NAMES => Creg Jan Anna Jay Jimmy Tom

# Same as:
NAMES=(${NAMES[@]} 'Jay' 'Jimmy' 'Tom')
# NAMES => Creg Jan Anna Jay Jimmy Tom

```

3.6.8. Iterating Over an Indexed Array

You can loop through array values directly or use indices to access each element.

```
names=('Jay' 'Joe' 'Jimmy')

# Direct element access
for name in "${names[@]"; do
    echo $name
done

# Index-based access
for i in "${!names[@]"; do
    printf "%s\t%s\n" "$i" "${names[$i]}"
done
```

3.6.9. Iterating Over an Associative Array

Looping over associative arrays is similar — just replace numeric indexes with string keys.

```
declare -A person
person=(["name"]="Jay" [age]=22 ["eye_color"]="blue")

# Loop over values
for value in "${person[@]"; do
    echo $value
done

# Loop over keys and values
for key in "${!person[@]"; do
    echo "$key ${person[$key]}"
done
```

3.6.10. Merging / Concatenating Arrays

You can merge multiple arrays into one using parameter expansion.

```
ARRAY1=(foo{1..2})    # => foo1 foo2
ARRAY2=(A..D)          # => A B C D
ARRAY3=(1..5)          # => 1 2 3 4 5

# Merge all into one
ARRAY4=("${ARRAY1[@]} ${ARRAY2[@]} ${ARRAY3[@]}")
# ARRAY4 => foo1 foo2 A B C D 1 2 3 4 5
```

```
# You can also use * instead of @
# ARRAY4=(${ARRAY1[*]} ${ARRAY2[*]} ${ARRAY3[*]})
```

3.6.11. Array Arguments to Functions

You can pass arrays to functions using **namerefs** (`local -n`) to make them accessible by name.

```
function extract()
{
    # Create a local reference to the array passed as $1
    local -n local_names=$1
    local index=$2
    echo "${local_names[$index]}"
}

names=('Jay' 'Joe' 'Jimmy')

extract names 2    # => Jimmy
```



Namerefs (using `local -n`) are supported in Bash 4.3 and later.

Chapter 4. Bash Conditions and Flow Control

Bash provides conditional expressions and control structures that allow your scripts to make decisions, evaluate input, and execute logic based on comparisons or patterns.

4.1. Numeric Conditions

Use these expressions to compare integer values. Bash supports both traditional `[ [ ... ] ]` comparisons and arithmetic-style `(( ... ))` expressions.

Condition	Description
<code>[[INTEGER1 -eq INTEGER2]]</code>	INTEGER1 is equal to INTEGER2
<code>[[INTEGER1 -ne INTEGER2]]</code>	INTEGER1 is not equal to INTEGER2
<code>[[INTEGER1 -lt INTEGER2]]</code>	INTEGER1 is less than INTEGER2
<code>[[INTEGER1 -le INTEGER2]]</code>	INTEGER1 is less than or equal to INTEGER2
<code>[[INTEGER1 -gt INTEGER2]]</code>	INTEGER1 is greater than INTEGER2
<code>[[INTEGER1 -ge INTEGER2]]</code>	INTEGER1 is greater than or equal to INTEGER2
<code>((INTEGER1 < INTEGER2))</code>	INTEGER1 is less than INTEGER2
<code>((INTEGER1 ≤ INTEGER2))</code>	INTEGER1 is less than or equal to INTEGER2
<code>((INTEGER1 > INTEGER2))</code>	INTEGER1 is greater than INTEGER2
<code>((INTEGER1 ≥ INTEGER2))</code>	INTEGER1 is greater than or equal to INTEGER2
<code>((INTEGER1 == INTEGER2))</code>	INTEGER1 is equal to INTEGER2
<code>((INTEGER1 != INTEGER2))</code>	INTEGER1 is not equal to INTEGER2



Use `(( ... ))` for cleaner syntax when comparing numbers — it's often more readable and doesn't require quoting variables.

4.2. String Conditions

Use these expressions when comparing strings or checking for patterns.

Condition	Description
<code>[[-z STR]]</code>	True if the string is empty
<code>[[-n STR]]</code>	True if the string is not empty
<code>[[STR == STR]]</code>	Strings are equal
<code>[[STR = STR]]</code>	Same as above (alternate syntax)
<code>[[STR < STR]]</code>	STR is less than the other (ASCII order)
<code>[[STR > STR]]</code>	STR is greater than the other (ASCII order)
<code>[[STR != STR]]</code>	Strings are not equal
<code>[[STR =~ REGEXP]]</code>	Matches regular expression pattern



Always use `[[ ... ]]` instead of `[ ... ]` when working with strings — it provides more robust syntax and supports pattern matching like `=~` .

4.3. File Conditions

Use these expressions to test properties of files, symbolic links, pipes, sockets, and devices.

Condition	Description
<code>[[-b FILE]]</code>	FILE exists and is a block device
<code>[[-c FILE]]</code>	FILE exists and is a character device
<code>[[-d FILE]]</code>	FILE exists and is a directory
<code>[[-e FILE]]</code>	FILE exists (regardless of type)
<code>[[-f FILE]]</code>	FILE exists and is a regular file
<code>[[-g FILE]]</code>	FILE exists and has the set-group-ID bit set

Condition	Description
<code>[[-G FILE]]</code>	FILE exists and is owned by the effective group ID
<code>[[-h FILE]]</code>	FILE exists and is a symbolic link (same as <code>-L</code>)
<code>[[-k FILE]]</code>	FILE exists and has the sticky bit set
<code>[[-L FILE]]</code>	FILE exists and is a symbolic link (same as <code>-h</code>)
<code>[[-N FILE]]</code>	FILE exists and was modified since last read
<code>[[-O FILE]]</code>	FILE exists and is owned by the effective user ID
<code>[[-p FILE]]</code>	FILE exists and is a named pipe (FIFO)
<code>[[-r FILE]]</code>	FILE exists and is readable by the user
<code>[[-s FILE]]</code>	FILE exists and has a size greater than zero
<code>[[-S FILE]]</code>	FILE exists and is a socket
<code>[[-t FD]]</code>	File descriptor FD is open and refers to a terminal
<code>[[-u FILE]]</code>	FILE exists and has the set-user-ID bit set
<code>[[-w FILE]]</code>	FILE exists and is writable by the user
<code>[[-x FILE]]</code>	FILE exists and is executable (or searchable if a directory)
<code>[[FILE1 -nt FILE2]]</code>	FILE1 is newer than FILE2
<code>[[FILE1 -ot FILE2]]</code>	FILE1 is older than FILE2
<code>[[FILE1 -ef FILE2]]</code>	FILE1 and FILE2 refer to the same file (same inode/device)

4.4. Compound Conditions

Use logical operators to combine multiple conditions.

Condition	Description
<code>[[condition1]]</code> <code>&& [[condition2]]</code>	True if both conditions are true (AND)

Condition	Description
<code>[[condition1]]</code> <code> [[condition2]]</code>	True if either condition is true (OR)
<code>[condition1 -a condition2]</code>	AND operator using single brackets (not recommended)
<code>[condition1 -o condition2]</code>	OR operator using single brackets (not recommended)



You can use either `[[ ... ]]` or `[ ... ]` for most tests. However, compound expressions using `-a` (AND) or `-o` (OR) **only work with** single square brackets and are prone to syntax errors.

Prefer using `[[ ... ]] && [[ ... ]]` and `[[ ... ]] || [[ ... ]]` for clarity and correctness.

4.5. More Conditions

These are additional conditional expressions you may encounter or use in more advanced scripting scenarios.

Condition	Description
<code>[[-o noclobber]]</code>	True if the shell option <code>noclobber</code> is enabled
EXPR	Evaluates whether an expression is true
<code>! EXPR</code>	Logical negation — true if the expression is false



The `[[ -o OPTION ]]` form is useful for checking shell behaviors, like whether overwriting files is disabled (`noclobber`), whether `allexport` is set, and so on.

4.6. if Statements

The `if` statement is one of the most important control structures in Bash. It allows you to execute commands conditionally — based on the success or failure (exit status) of a command or expression.

4.6.1. Basic Syntax

```
if CONDITION; then
    commands
fi
```

If the condition is true (i.e., the command returns an exit status of 0), the code inside the then block is executed.

4.6.2. if...else

```
if CONDITION; then
    commands_if_true
else
    commands_if_false
fi
```

Use else to define an alternate path when the condition is false.

4.6.3. if...elif...else

You can chain multiple conditions using elif.

```
if [[ $value -eq 1 ]]; then
    echo "Value is 1"
elif [[ $value -eq 2 ]]; then
    echo "Value is 2"
else
    echo "Value is something else"
fi
```

4.6.4. Using [[...]] vs [...] vs ...

- Use [[...]] for string tests, pattern matching, and complex conditions.
- Use ((...)) for arithmetic comparisons.
- [...] is POSIX-compatible but more limited and error-prone.

```
if [[ "$name" == "Jay" ]]; then
    echo "Hello Jay"
fi

if (( 10 > 5 )); then
    echo "Math works"
fi
```



Every if must be closed with fi. Also, remember that Bash uses exit status (0 = success, non-zero = failure) to evaluate truth.

4.6.5. if with Command Execution

You can use `if` to check the result of any command — not just expressions.

```
if grep "error" logfile.txt > /dev/null; then
    echo "Found an error in the log"
else
    echo "No errors found"
fi
```



Redirecting output (e.g., `> /dev/null`) is often used to silence command output in conditions.

4.6.6. Nested if Statements

You can place an `if` block inside another `if` block to evaluate multiple conditions in sequence.

```
if [[ "$user" == "admin" ]]; then
    if [[ "$access" == "full" ]]; then
        echo "Admin with full access"
    else
        echo "Admin with limited access"
    fi
else
    echo "Not an admin"
fi
```



Nested `if` statements can make your script harder to read — use them carefully. Consider using `elif` or logical operators when possible.

4.6.7. Compound Conditions with `&&` and `||`

Instead of nesting multiple `if` statements, you can use logical operators to combine them in a single line.

```
if [[ "$user" == "admin" && "$access" == "full" ]]; then
    echo "Admin with full access"
fi
```

- `&&` (AND): both conditions must be true
- `||` (OR): at least one condition must be true

```
if [[ "$user" == "admin" || "$user" == "support" ]]; then
    echo "Privileged user"
fi
```

4.6.8. Grouping with Double Brackets

When combining multiple conditions, always use `[[ ... ]]` to take advantage of grouping and short-circuit logic.

```
if [[ "$status" == "ok" && ( "$role" == "user" || "$role" == "admin" ) ]]; then
    echo "Access granted"
fi
```



Grouping with parentheses requires a space between `(` and the condition inside, and the whole condition must be within `[[ ... ]]`.

4.7. The **test** Command

The `test` command is used to evaluate expressions such as comparisons and file checks. It's often written using square brackets (`[ ... ]`), which is just a shorthand for `test`.

4.7.1. Basic Syntax

```
test EXPRESSION
# or using the shorthand
[ EXPRESSION ]
```

If the expression is true, `test` returns exit status `0`; otherwise, it returns a non-zero exit status.

4.7.2. Examples

```
# Check if a file exists
test -e /etc/passwd
[ -e /etc/passwd ]

# Compare numbers
test 5 -gt 3
[ 5 -gt 3 ]

# Compare strings
test "$name" = "Jay"
[ "$name" = "Jay" ]
```

There must be **spaces** around the brackets and between arguments. For example:



- `[ "$name" = "Jay" ]`
- `["$name"="Jay"]` (syntax error)

4.7.3. Why Use `test`?

- POSIX-compliant (portable across different shells)
- Available even in minimal shell environments
- Useful when writing scripts that don't rely on Bash-specific features



For more advanced comparisons (like regex matching), prefer `[[ ... ]]` which is a Bash extension with better quoting rules and features.

4.8. Advanced if-then Features

Bash gives you several powerful syntaxes to enhance `if` conditions with subshells, arithmetic, and extended test features.

- **Subshells with parentheses** `( ... )` You can execute a command in a subshell and use its result in your condition. Useful for isolating environment changes or combining multiple commands.
- **Arithmetic with double parentheses** `(( ... ))` Enables arithmetic comparisons and calculations. This makes numeric logic simpler and more readable.



For more on math in Bash, see the section on performing arithmetic operations.

- **Advanced tests with double brackets** `[[ ... ]]` Offers extended syntax for string comparison, regex matching (`=~`), and better quoting behavior than `[ ... ]`.



Prefer using `[[ ... ]]` and `(( ... ))` over single brackets or external commands like `test` for more consistent and safer condition handling in modern Bash.

4.9. case Statement

The `case` statement lets you compare one value against multiple patterns. It's ideal when you want to handle different inputs or conditions without writing several `if-elif` blocks.

Here's the basic structure:

```

case WORD in
    PATTERN1)
        commands
        ;;
    PATTERN2)
        commands
        ;;
    *)
        fallback_commands
        ;;
esac

```

The patterns can include wildcards like `*`, and the fallback case is written as ``)`.

Let's say you're writing a script that accepts a user action like `start`, `stop`, or `restart`. A case statement makes this much easier to read than nested if blocks.

```

read -p "Choose [start|stop|restart]: " action

case "$action" in
    start)
        echo "Starting service..."
        ;;
    stop)
        echo "Stopping service..."
        ;;
    restart)
        echo "Restarting service..."
        ;;
    *)
        echo "Unknown action: $action"
        ;;
esac

```

You can also use case to process file types. Here's how to identify file types based on their extension:

```

file="notes.txt"

case "$file" in
    *.txt)
        echo "It's a text file"
        ;;
    *.jpg | *.png)
        echo "It's an image file"
        ;;
    *)

```

```
    echo "File type not recognized"
;;
esac
```



Use the `|` symbol to match multiple patterns in one case. You can also use glob-style matching (like `.sh` or `file-`) inside each pattern.



Always quote the variable used in case (e.g., `"$file"`) to avoid word-splitting or unexpected matches when it contains spaces.

4.10. Loops That Use Conditions

Some Bash loops depend directly on the result of a condition — they keep running while the condition is true, or until it becomes true.

4.10.1. while

The `while` loop repeats as long as the condition remains true.

```
counter=1

while [[ $counter -le 3 ]]; do
    echo "Count: $counter"
    ((counter++))
done
```

The loop stops when the condition fails — in this case, when `$counter` becomes greater than 3.

4.10.2. until

The `until` loop is the opposite of `while` — it runs until the condition becomes true.

```
counter=1

until [[ $counter -gt 3 ]]; do
    echo "Count: $counter"
    ((counter++))
done
```

This keeps running **while** the condition is false, and stops **once it becomes true**.

Chapter 5. Bash Loops

Loops allow you to run a set of commands repeatedly. Bash provides several loop types, each suited for different use cases.

5.1. Basic for Loop

The for loop is often used to iterate over a list of items. In its most common form, it loops through the results of a glob or a word list.

```
for i in /etc/*; do
    echo $i
done
```

You can also write the loop using a line-break style, which works identically:

```
for i in /etc/*
do
    echo $i
done
```

This loop prints every file or directory inside /etc.

5.2. C-like for Loop

Bash also supports a C-style for loop, which gives you full control over initialization, condition, and increment.

```
for ((i = 0 ; i < 100 ; i++)); do
    echo $i
done
```

This is equivalent to:

```
for ((i = 0 ; i < 100 ; i++))
do
    echo $i
done
```

Use this form when working with numeric ranges or indexed iteration.

5.3. Using Ranges

Brace expansion provides a simple way to loop through a sequence of numbers.

```
for i in {1..10}; do
    echo "Number: $i"
done
```

You can also specify a step value:

```
for i in {5..50..5}; do
    echo "Number: $i"
done
```

This prints: 5, 10, 15, ..., 50



Brace expansion happens **before** the loop runs. If you're using variables to define the range (like {1..\$limit}), it won't work — you'll need to use a C-style loop or `seq`.

5.4. Bash while Loop

The `while` loop repeats commands **as long as a condition remains true**. It's useful when you're not sure how many times a loop should run and want to stop based on logic instead of a fixed list.

5.4.1. Counting Up

```
i=1
while [[ $i -lt 4 ]]; do
    echo "Number: $i"
    ((i++))
done
```

This prints numbers 1 through 3. The loop runs while `$i` is less than 4.

5.4.2. Counting Down

```
i=3
while [[ $i -gt 0 ]]; do
    echo "Number: $i"
    ((i--))
done
```

This prints numbers 3, 2, 1 — and stops when `$i` is no longer greater than 0.

5.4.3. while true Loop

If you want a loop that runs forever (until manually broken), you can use `while true` or `while :`.

```
# Long form
while true; do
    # TODO
done

# Shorthand
while ;; do
    # TODO
done
```

Use `break` to exit the loop when a condition is met inside.

5.4.4. Reading Files with while

The `while` loop is commonly used to read a file line by line.

```
# Using a pipe
cat file.txt | while read line; do
    echo $line
done
```

This works but has a subshell caveat — any variables modified inside the loop won't persist outside.

To avoid that, use input redirection instead:

```
while read line; do
    echo $line
done < "/path/to/txt/file"
```

This version runs in the current shell, so variables inside the loop are retained after it ends.

NOTE

`read` reads one line at a time and splits input on whitespace. Use `IFS=` before `read` if you want to preserve leading/trailing spaces:

```
while IFS= read -r line; do
    echo "$line"
done < file.txt
```

5.5. until Loop

The `until` loop is like the opposite of `while`: it runs **until** the condition becomes true.

```
count=0
until [ $count -gt 10 ]; do
    echo "$count"
    ((count++))
done
```

This loop runs as long as `$count` is **not** greater than 10 — that is, from 0 to 10.

You can also count down:

```
count=10
until [ $count -eq 0 ]; do
    echo "$count"
    ((count--))
done
```

The loop stops once `$count` reaches 0.



Unlike `while`, `until` loops continue **while the condition is false**, and exit when it becomes true.

5.6. continue Statement

The `continue` statement skips the rest of the current loop iteration and jumps to the next one.

```
for number in $(seq 1 3); do
    if [[ $number == 2 ]]; then
        continue
    fi
done
```

```
fi
echo "$number"
done
```

This prints: 1 3

The number 2 is skipped because of the `continue`.

The `seq` command is a convenient way to generate a range: `seq 1 3` produces 1 2 3.

5.7. break Statement

The `break` statement exits the loop immediately, skipping all remaining iterations.

```
for number in $(seq 1 3); do
    if [[ $number == 2 ]]; then
        break
    fi
    echo "$number"
done
```

This prints: 1

As soon as the loop sees 2, it stops entirely.



Both `break` and `continue` can be used in any loop type: `for`, `while`, `until`, and even `select`.

5.8. Additional Notes on Loops

You've now seen all the core loop types in Bash — `for`, `while`, `until`, and `select` — along with control statements like `break` and `continue`. Below are some extra tips and features that may come in handy when your scripts get more advanced.

5.8.1. Nested Loops

You can place one loop inside another to work with two dimensions (e.g., rows and columns, files and lines, etc.).

```
for i in {1..2}; do
    for j in {A..B}; do
        echo "$i$j"
    done
done
```

```
done
```

This prints: 1A, 1B, 2A, 2B.

5.8.2. Looping Over Arrays with `read -a`

You can use `read -a` to read words into an array and then loop through the array.

```
read -a colors <<< "red green blue"

for color in "${colors[@]}; do
    echo $color
done
```

This is useful when working with space-separated input.

Breaking or Continuing Multiple Levels

If you're in nested loops, you can use `break N` or `continue N` to jump out of multiple levels at once.

```
for i in {1..3}; do
    for j in {1..3}; do
        if [[ $j -eq 2 ]]; then
            break 2 # Exits both loops
        fi
        echo "$i $j"
    done
done
```

5.8.3. Looping Over Command Output Safely

To loop over the output of a command (e.g., `find`), use a **process substitution** and read with `IFS=` to preserve whitespace.

```
while IFS= read -r file; do
    echo "$file"
done < <(find . -type f)
```

Avoid using `for file in $(command)` — it breaks when filenames contain spaces.

5.8.4. Creating an Infinite Loop with a Delay

Sometimes you want a loop to run continuously with a pause between runs. Here's a common pattern:

```
while true; do
    echo "Checking system..."
    sleep 10
done
```

Use `sleep` to control how frequently the loop runs.

5.8.5. Performance and Portability Tip

The `seq` command is useful for quick loops, but it's an external command — which means it spawns a subprocess.

```
for i in $(seq 1 100000); do
    ...
done
```

In large-scale scripts, a Bash-native `for ...` loop performs better:

```
for ((i = 1; i <= 100000; i++)); do
    ...
done
```

Use native Bash syntax (`...`, `[...]`, brace expansion) wherever possible — it's faster and doesn't rely on external commands.

5.9. The select Loop

The `select` loop is a Bash-specific feature designed to simplify interactive menus. It automatically displays a numbered list of choices and waits for the user to make a selection.

It's most useful in scripts that require basic user input without needing to build a menu system from scratch.

5.9.1. Basic Syntax

```
select variable in list; do
    commands
    break
done
```

- `variable` is assigned the value of the selected item.
- `list` is a space-separated list of options.

- The loop continues until you manually break it using `break` or `exit`.

5.9.2. Menu Example

```
select fruit in apple banana cherry; do
    echo "You selected $fruit"
    break
done
```

When run, this prints:

```
apple
banana
cherry
```

Typing 2 selects banana, assigns it to `fruit`, and prints:

5.9.3. Handling Invalid Input

If the user enters an invalid number (like 0 or something out of range), the variable will be empty. You can check for this and continue prompting:

```
select option in "Start" "Stop" "Restart"; do
    if [[ -n "$option" ]]; then
        echo "You chose: $option"
        break
    else
        echo "Invalid selection"
    fi
done
```

5.9.4. Changing the Prompt

By default, `select` uses `PS3` as the prompt. You can customize it to give better context to the user.

```
PS3="Choose a service action: "
select action in Start Stop Restart; do
    echo "Action: $action"
    break
done
```

The `select` loop is rarely used in scripts that run automatically, but it's a great tool for building simple, interactive

menus.

Chapter 6. Bash Functions

Functions in Bash are reusable blocks of code you can call by name. They help organize scripts, eliminate repetition, and create readable logic structures.

6.1. Defining Functions

There are two ways to define a function. Both are valid and work the same way.

```
greet() {  
    echo "hello $1 $2"  
}  
  
# Alternate syntax  
function greet() {  
    echo "hello $1 $2"  
}  
  
# Calling the function  
greet "John" "Paul"
```

Arguments passed to the function are accessed using \$1, \$2, etc., just like positional parameters in scripts.

6.2. Returning Values

Bash functions don't use return to send back data. Instead, use echo and **command substitution** to capture output.

```
add_numbers() {  
    local result=$(( $1 + $2 ))  
    echo $result  
}  
  
# Storing the result  
sum="$(add_numbers 1 2)"  
echo "Result: $sum"
```



The return keyword in Bash can only return **exit status codes** (0–255), not data.

6.3. Raising Errors with return

You can use return to indicate success or failure using exit codes.

```

myfunc() {
    return 1 # non-zero means failure
}

if myfunc; then
    echo "success"
else
    echo "failure"
fi

```

This allows your script to react to different outcomes, like testing the result of a condition.

6.4. Recursive Functions

A function can call itself — just be sure to define a condition to stop the recursion, or it will loop forever.

```

countdown() {
    if [[ $1 -le 0 ]]; then
        echo "Done!"
        return
    fi
    echo "$1"
    countdown $(( $1 - 1 ))
}

countdown 3

```

This prints:

```

3
2
1
Done!

```



Bash doesn't optimize recursion. Use it sparingly — deep recursion may cause stack overflows or performance issues.

6.5. Fork Bomb

A fork bomb is a self-replicating function that quickly consumes system resources by endlessly spawning itself. It's used for demonstration or malicious purposes — **never run this on a real system**.

```
:(){ :|:& };;:
```

This creates a function `:` that calls itself twice in the background, over and over.



Running a fork bomb will freeze your system by consuming all process slots. Only test it inside an isolated VM or container — if at all.

6.6. More on Functions

Functions in Bash are flexible, but there are a few important behaviors and lesser-known features worth noting.

6.6.1. Variables Are Global by Default

Unless you explicitly use `local`, any variable you define inside a function will affect the global shell environment.

```
greet() {  
    name="Jay"      # This is global  
}  
  
greet  
echo "$name"      # Prints: Jay
```

To avoid accidental overwrites, use `local`:

```
greet() {  
    local name="Jay"  
    echo "$name"  
}
```

6.6.2. Unsetting or Overwriting Functions

You can redefine a function by simply writing a new version with the same name. To remove a function from the shell, use:

```
unset -f greet
```

This is useful for cleaning up function definitions in long scripts or interactive sessions.

6.6.3. Listing Functions

To see a list of all defined functions in your current shell:

```
declare -F
```

This prints the names (and optionally locations) of every declared function.

6.6.4. Valid Function Names

Function names should start with a letter or underscore and avoid special characters. Avoid using names that conflict with built-in commands (like `test`, `echo`, or `read`), unless you intend to override them.

```
# valid
hello_world() { echo "Hi"; }

# not recommended
test() { echo "bad idea"; } # Conflicts with the `test` command
```

6.6.5. Exporting Functions to Subshells (Bash 4+)

You can export a function to be used by child shells or other scripts that run in the same environment.

```
say_hello() {
    echo "Hello from parent shell"
}

export -f say_hello
bash -c 'say_hello' # Works because it's exported
```



Only Bash version 4.0 or later supports function exporting this way.

Chapter 7. Bash Arithmetic Operations

Bash supports a rich set of arithmetic and bitwise operators. You can use them inside arithmetic expansion `$...`, or with conditional structures like `if`, `while`, and `let`.

The table below summarizes all the available arithmetic operators in Bash:

Arithmetic Operator	Description
<code>+</code> , <code>-</code>	Addition and subtraction
<code>*</code> , <code>/</code> , <code>%</code>	Multiplication, division, and remainder (modulo)
<code>**</code>	Exponentiation
<code><=</code> , <code>>=</code> , <code><</code> , <code>></code>	Less than or equal, greater than or equal, less than, and greater than
<code>==</code> , <code>!=</code>	Equality (equal to) and inequality (not equal to)
<code>=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> , <code>+=</code> , <code>-=</code> , <code><<=</code> , <code>>>=</code> , <code>&=</code> , <code>^=</code> , <code> =</code>	Assignment operators
<code>id++</code> , <code>id--</code>	Variable post-increment and post-decrement
<code>++id</code> , <code>--id</code>	Variable pre-increment and pre-decrement
<code>&&</code>	Logical AND
<code> </code>	Logical OR
<code>-</code> , <code>+</code>	Unary minus, plus
<code>!</code> , <code>~</code>	Logical and bitwise negation
<code><<</code> , <code>>></code>	Left and right bitwise shifts
<code>&</code>	Bitwise AND
<code>^</code>	Bitwise XOR
<code> </code>	Bitwise OR
<code>expression ? expression : expression</code>	Conditional ternary operator



All arithmetic operations in Bash must be performed in a valid arithmetic context — usually inside `...` or with the `let` command.

7.1. Bash Arithmetic Operations

7.1.1. Using `let` to Perform Arithmetic Operations

The `let` command is a built-in in Bash used to evaluate arithmetic expressions. It allows you to assign the result of a calculation to a variable or perform inline operations.

The basic syntax is:

```
let <arithmetic expression>
```

If you don't quote the expression, you must **not use spaces** between operators and operands. Quoting allows you to use spaces and makes expressions more readable.

```
# Without quotes, no spaces allowed
let result=6+4
echo $result    # => 10

# With quotes, you can add spaces
let "result = 4 + 4"
echo $result    # => 8
```

You can also use shorthand operators like `+=` or `=` to modify values.

```
# Increment result by 1
let result++
echo $result    # => 9

# Multiply two numbers
let "result = 4 * 5"
echo $result    # => 20
```

You can pass values using variables, such as command-line arguments:

```
let "result = $1 + 30"
echo $result    # 30 plus the first argument
```



`let` is simple and readable, but it doesn't support floating-point math. For that, you'll need external tools like `bc` or `awk`.

7.1.2. Using `expr` to Perform Arithmetic Operations

The `expr` command is an external program (not a Bash built-in) that can perform arithmetic operations and string evaluations. Since it spawns a new process, it's generally **slower** than Bash-native methods like `let` or `$...`, especially inside loops.



Because `expr` is external and may behave differently on different systems, it's not recommended for performance-critical or portable scripts.

Basic Syntax

```
expr item1 operator item2
```

Unlike `let`, you **must use spaces** between the expression items, and **should not use quotes** around the expression.

```
expr 7 + 5      # => 12
expr 3+2        # => 3+2 (no spaces = not evaluated)
expr "6 + 3"    # => 6 + 3 (quoted = not evaluated)
```

Multiplication and Special Characters

Some symbols (like `*`) have special meaning in Bash, so you must **escape them with a backslash**.

```
expr 5 \* 3     # => 15
```

Modulus and Other Operators

You can also perform modulus, subtraction, or any other integer operation supported:

```
expr 15 % 2     # => 1
```

Capturing the Result

Use command substitution to store the result of `expr` in a variable:

```
result=$(( expr 15 - 2 ))
echo $result      # => 13
```



Always include spaces between the operands and operator. Bash treats `expr 1+1` and `expr "1 + 1"` as literal strings, not calculations.

7.1.3. Using ... for Arithmetic Operations

The double parentheses syntax ... is the most recommended way to perform arithmetic in Bash. It is **built into the shell**, faster, more flexible, and does not require quoting or escaping special characters.

Syntax

```
$(( arithmetic expression ))
```

This is known as **arithmetic expansion**.

Examples

```
# Basic usage
result=$(( 6 + 5 ))
echo $result    # => 11

# No need to use `$` inside arithmetic expressions
result2=$(( result + 3 ))
echo $result2   # => 14

# You can still use `$` if you prefer
result3=$(( $result + 4 ))
echo $result3   # => 15

# Pre/post increment without `$`
(( result3++ ))
echo $result3   # => 16

# Augmented assignment
(( result3 += 4 ))
echo $result3   # => 20

# Multiplication without escaping `*`
result=$(( 4 * 5 ))
echo $result    # => 20
```



Inside ..., variable names **don't need the \$** prefix. Both `x + 1` and `$x + 1` work, but using the plain name is cleaner.



There is also an older syntax using square brackets: `$( [ expression ] )` — but this is deprecated and **should not be used**.

declare -i for integer math

You can declare a variable as an integer with `declare -i`, which allows arithmetic evaluation automatically:

```
declare -i sum
sum=5+3
echo $sum # => 8
```

7.2. Floating Point Arithmetic with bc

*# Bash doesn't support floating-point arithmetic directly.
To perform calculations with decimals, use the `bc` command.*

```
result=$(echo "scale=2; 5.4 + 3.3" | bc)
echo "$result" # => 8.7
```

```
# multiplication
result=$(echo "3.2 * 2.1" | bc)
echo "$result" # => 6.72
```

```
# division
result=$(echo "scale=3; 10 / 3" | bc)
echo "$result" # => 3.333
```



scale=2 sets the number of decimal places in the result. Without it, bc defaults to zero for division.

7.2.1. Floating Point Arithmetic with awk

awk can also handle decimals

```
result=$(awk 'BEGIN { print 10.5 / 2.5 }')
echo "$result" # => 4.2
```

```
# more complex expression
result=$(awk 'BEGIN { print (3.5 + 1.2) * 2 }')
echo "$result" # => 9.4
```

7.3. Arithmetic with Arrays

```
# summing array values
numbers=(3 7 2)
sum=0
```

```
for n in "${numbers[@]}; do
    (( sum += n ))
done

echo "Sum: $sum" # => 12
```

7.4. Preventing Division by Zero

```
a=10
b=0

if [[ $b -eq 0 ]]; then
    echo "Division by zero is not allowed"
else
    echo $(( a / b ))
fi
```



Division by zero will crash your script with an error. Always check the denominator before dividing.

Chapter 8. Bash Parameter Expansion

Parameter expansion in Bash gives you powerful tools to manipulate variable content directly within the shell—without needing to rely on external commands. It's commonly used for string trimming, pattern replacement, conditional value assignment, and more. These operations are lightweight and efficient because they are handled internally by the shell.

8.1. Syntax

These forms let you modify variables by removing or replacing portions of their contents based on patterns. You can strip prefixes or suffixes, or perform inline substitutions using search-and-replace logic.

Syntax	Description
<code>\${parameter%suffix}</code>	Remove suffix
<code>\${parameter#prefix}</code>	Remove prefix
<code>\${parameter%%suffix}</code>	Remove longest matching suffix
<code>\${parameter##prefix}</code>	Remove longest matching prefix
<code>\${parameter/pattern/string}</code>	Replace first pattern match with string
<code>\${parameter//pattern/string}</code>	Replace all pattern matches with string
<code>\${parameter/%pattern/string}</code>	Replace suffix pattern with string
<code>\${parameter/#pattern/string}</code>	Replace prefix pattern with string

8.2. Substrings

You can extract specific portions of a string using a start position and length, similar to substring functions in other programming languages. Bash also allows indexing from the end of the string using negative numbers.

Syntax	Description
<code>\${FOO:0:3}</code>	Substring (position, length)
<code>\${FOO:(-3):3}</code>	Substring from the right

8.3. Length

You can retrieve the length of a variable's value—particularly useful when validating input or processing strings programmatically.

Syntax	Description
<code>\${#FOO}</code>	Length of <code>\$FOO</code>

8.4. Default Values

These forms allow you to define fallback behaviors when a variable is unset or null. You can provide a default value, assign one on the fly, or even display an error if a variable isn't set.

Syntax	Description
<code>\${FOO:-val}</code>	<code>\$FOO</code> , or <code>val</code> if unset
<code>\${FOO:=val}</code>	Set <code>\$FOO</code> to <code>val</code> if unset
<code>\${FOO:+val}</code>	<code>val</code> if <code>\$FOO</code> is set
<code>\${FOO:?message}</code>	Show message and exit if <code>\$FOO</code> is unset

8.5. Substitution

```
echo ${food:-Cake}    #=> $food or "Cake"
STR="/path/to/foo.cpp"
echo ${STR%.cpp}      # /path/to/foo
echo ${STR%.cpp}.o    # /path/to/foo.o
echo ${STR%/*}        # /path/to

echo ${STR##*.}       # cpp (extension)
echo ${STR##*/}       # foo.cpp (basepath)

echo ${STR#*/}        # path/to/foo.cpp
echo ${STR##*/}       # foo.cpp

echo ${STR/foo/bar}   # /path/to/bar.cpp
```

8.6. Slicing

```
name="John"
echo ${name}          # => John
```

```

echo ${name:0:2}      # => Jo
echo ${name::2}       # => Jo
echo ${name::-1}      # => Joh
echo ${name:(-1)}     # => n
echo ${name:(-2)}     # => hn
echo ${name:(-2):2}   # => hn

length=2
echo ${name:0:length} # => Jo

```

8.7. Basepath & Dirpath

```

SRC="/path/to/foo.cpp"
BASEPATH=${SRC##*/}
echo $BASEPATH # => "foo.cpp"

DIRPATH=${SRC%$BASEPATH}
echo $DIRPATH  # => "/path/to/"

```

8.8. Transform

```

STR="HELLO WORLD!"
echo ${STR,}    # => hELLO WORLD!
echo ${STR,,}   # => hello world!

STR="hello world!"
echo ${STR^}    # => Hello world!
echo ${STR^^}   # => HELLO WORLD!

ARR=(hello World)
echo "${ARR[@],}" # => hello world
echo "${ARR[@]^}" # => Hello World

```

Chapter 9. Command Line Chaining

Bash allows you to combine multiple commands on a single line, controlling how and when each command runs. This is called **command chaining**. It's useful when you want to execute commands conditionally, sequentially, or in the background.

9.1. & – Run in the Background

Run a command in the background so your terminal stays available for other input:

```
sleep 10 &
```

This runs `sleep` in the background and immediately returns control to the terminal. The job will still complete in the background.

9.2. && – Run If Previous Succeeds

Use `&&` when you want the second command to run **only if** the first one is successful (exit status 0):

```
mkdir project && cd project
```

If `mkdir` creates the directory successfully, then `cd` into it. If `mkdir` fails (e.g. directory exists), `cd` is skipped.

9.3. || – Run If Previous Fails

This runs the second command **only if** the first one fails (non-zero exit status):

```
cd missing_dir || echo "Directory not found"
```

Here, if the directory doesn't exist and `cd` fails, it prints an error message.

9.4. ; – Always Run Next Command

The semicolon executes all commands in sequence, regardless of the success or failure of the previous one:

```
echo "Start" ; invalid_command ; echo "Still running"
```

Even if `invalid_command` fails, the last `echo` will still execute.

9.5. ! – Negate Exit Status

Use ! to invert a command's result. A successful command becomes a failure and vice versa:

```
! grep "text" file.txt
```

If grep finds "text", ! treats it as failure. If it doesn't find it, ! treats it as success.

9.6. && || – Simulate If-Else

Combining && and || mimics if-else logic:

```
check_something && echo "OK" || echo "FAILED"
```

- If check_something succeeds, prints OK.
- If it fails, prints FAILED.

Use grouping to avoid fall-through when the middle command fails:

```
check_something && { echo "OK"; } || echo "FAILED"
```

9.7. | – Pipe Output to Another Command

The pipe takes output from one command and feeds it to the next:

```
ls -l | grep ".txt"
```

This lists files and filters only .txt ones using grep.

9.8. >, >>, < – Redirect Input and Output

- > redirects and overwrites a file.
- >> appends to a file.
- < reads input from a file.

```
echo "Line" > file.txt      # Overwrites
echo "Another" >> file.txt  # Appends
wc -l < file.txt           # Reads input
```

9.9. \ – Continue Command on New Line

Split long commands across multiple lines for readability:

```
echo "This is a long line \  
that continues here"
```

The backslash tells Bash to treat the next line as a continuation.

9.10. () – Subshell Execution

Parentheses group commands in a **subshell**. Variables inside won't affect the outer shell:

```
(count=5; echo $count)  
echo $count # Not defined
```

9.11. {} – Grouping in Current Shell

Curly braces group commands in the current shell and allow redirection to apply to all of them:

```
{ echo "First"; echo "Second"; } > all_output.txt
```

All grouped commands output to the same file.



Use grouping when you want to treat several commands as a block, especially in loops or redirection.



When chaining commands, always consider exit statuses and context. For example, `&&` and `|` depend on how a command exits, so chaining them without checking return values can lead to unexpected behavior.

Chapter 10. Command Substitution

Command substitution lets you capture the output of a command and use it as part of another command or assign it to a variable. This is especially useful when scripting, as it allows you to embed live command results directly into your logic.

10.1. Syntax

There are two ways to perform command substitution in Bash:

```
$(command)    # Preferred modern syntax
`command`     # Legacy syntax (backticks)
```

The `$(...)` syntax is easier to read and nest, and is preferred for clarity.

10.2. Basic Example

```
today=$(date +%F)
echo "Today's date is $today"
```

This runs the `date +%F` command and stores the result (e.g. 2025-06-22) in the variable `today`.

Without command substitution, this would assign the literal string:

```
today=date +%F
echo "Today's date is $today"  # Outputs: Today's date is date +%F
```

10.3. Assigning Values Dynamically

You can use command substitution to assign system-generated or calculated values to variables.

```
count=$(ls | wc -l)      # Count number of items in current directory
dirsize=$(du -sh /home)  # Get human-readable size of /home directory
```

10.4. Using Output as Arguments

You can pass the result of a command directly as an argument to another command:

```
grep "foo" $(find . -name '*.txt')  # Search "foo" in all .txt files
```

```
tar -cvzf $(date +%F).tar.gz /mydir    # Backup folder with today's date
```



Command substitution strips trailing newlines from the output. If you need to preserve exact formatting (including newlines), consider using `readarray` or process substitution instead.

Chapter 11. Process Substitution

Process substitution allows you to use the output or input of a command as if it were a file. This makes it possible to pass the output of one command directly into another where a file is normally expected.

11.1. Syntax

```
<(command)    # Treats output as a file (used as input)
>(command)    # Treats input as a file (receives output)
```

The `<(...)` form is the most commonly used. It creates a temporary, system-managed file descriptor that the receiving command can read from.

11.2. Comparing Files

You can compare the output of commands without writing temporary files:

```
diff <(sort data.txt) <(sort backup-data.txt)
```

This compares the sorted contents of two files directly using `diff`, without needing to create intermediate files.

11.3. Comparing Command Output

```
diff <(ls /data) <(ls /data-backup)
```

Here, `ls` output from two directories is compared using `diff`, treating both outputs as if they were files.

11.4. Searching Through Output

```
grep 'error' <(cat /var/log/syslog /var/log/apache2/error.log)
```

This searches for the word "error" across multiple log files. The `cat` output is treated as a single input file to `grep`.

11.5. Sorting and Saving Output

```
sort <(ps aux) > processes.sorted
```

The `ps aux` output is sorted and redirected to a file. No intermediate file is created to hold the raw process list.

11.6. Opening Multiple Files

```
vim <(cat data.txt new.txt)
```

You can use process substitution to open the combined content of multiple files in `vim` as if they were a single file.

11.7. Transferring Compressed Files Over SSH

```
tar -cf >(ssh remote_server tar xf -)
```

This creates a tar archive and streams it over SSH to a remote machine, where it's extracted — all in one command.

11.8. Benefits

- **No temporary files:** Bash handles the creation and cleanup of the file descriptors automatically.
- **Cleaner pipelines:** Makes your commands and scripts more readable by avoiding extra redirection logic.
- **Works with commands expecting filenames:** Lets you "trick" tools into reading from or writing to the output of other commands.



Process substitution requires a shell that supports it — like `bash` or `zsh`. It won't work in POSIX `sh`.

Chapter 12. Bitwise Operators in Bash

Bitwise operators allow you to manipulate data at the binary level — flipping, shifting, or combining bits. These operators are useful in scripts where you work with system flags, low-level network operations, or data processing that benefits from binary logic.

12.1. What Are Bits and Bytes?

Before diving into the operators, it's important to understand how numbers are represented in binary. Each binary digit (bit) holds a value of either 0 or 1.

A byte is made up of 8 bits and can represent numbers from 0 to 255. For example:

```
Decimal: 13
Binary:  00001101
```

Each position in a binary number represents a power of 2, increasing from right to left.

12.2. Supported Bitwise Operators in Bash

Bash supports six bitwise operators:

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR (exclusive OR)
~	Bitwise NOT
<<	Left Shift
>>	Right Shift

All bitwise operations in Bash are performed using the `$...` arithmetic expansion syntax.

12.3. Bitwise AND (&)

The AND operator compares two numbers bit by bit. It only sets a bit to 1 if **both** bits are 1.

```
a=15      # 1111
b=7        # 0111
result=$((a & b))
```

```
echo $result # Output: 7
```

The binary result of `1111 & 0111` is `0111`, which is 7 in decimal.

12.4. Bitwise OR (`|`)

The OR operator sets a bit to 1 if **either** bit is 1.

```
a=15      # 1111
b=7       # 0111
result=$((a | b))
echo $result # Output: 15
```

`1111 | 0111` results in `1111`, or 15 in decimal.

12.5. Bitwise XOR (`^`)

The XOR operator sets a bit to 1 only if the bits are different.

```
a=15      # 1111
b=7       # 0111
result=$((a ^ b))
echo $result # Output: 8
```

`1111 ^ 0111` equals `1000`, or 8 in decimal.

12.6. Bitwise NOT (`~`)

The NOT operator flips every bit of the number. In Bash (and most shells), numbers are signed 32-bit integers. So flipping all bits of a number results in a negative value.

```
a=15      # 00000000 00000000 00000000 00001111
result=$((~a))
echo $result # Output: -16
```

Flipping all bits of 15 yields -16 because of two's complement representation.

12.7. Bitwise Left Shift (`<<`)

This operator shifts all bits to the left, effectively multiplying the number by 2 for each shift.

```
a=8      # 1000
result=$((a << 2))
echo $result # Output: 32
```

1000 << 2 becomes 100000, which is 32 in decimal.

12.8. Bitwise Right Shift (>>)

This operator shifts bits to the right, effectively dividing the number by 2 for each shift.

```
a=32     # 100000
result=$((a >> 2))
echo $result # Output: 8
```

100000 >> 2 becomes 1000, which is 8 in decimal.

12.9. Practical Uses of Bitwise Operators

- Toggle feature flags in configuration variables
- Parse protocol headers or status registers
- Perform quick integer multiplications/divisions
- Create masks for selective bit extraction



While not used daily in typical scripting tasks, bitwise operators become powerful tools in performance-sensitive scripts or when dealing with binary data.

Chapter 13. Quoting in Bash

Quoting can be tricky for new Linux users. Bash uses special characters like `$`, `,`, and backticks (``` `\`` ```) to represent variables, wildcards, and command substitution. But what if you want to display these characters *literally, without triggering their special meanings?

That's where quoting comes in. Bash provides three main ways to control how special characters behave:

- Double quotes: `"..."` allow some interpretation.
- Single quotes: `'...'` treat everything literally.
- Backslashes: `\` escape specific characters.

13.1. Using Double Quotes

Double quotes `"..."` allow Bash to interpret some but not all special characters. Variables and command substitutions are expanded, but wildcards (`*`) and filename globbing are not.

13.1.1. Expanding Variables

This command will expand the `$USER` variable to your current username:

```
echo "$USER"
```

Output:

```
traw
```

13.1.2. Command Substitution Inside Double Quotes

```
echo "The time is `date`."
```

Output:

```
The time is Wed Feb 28 22:44:37 SAST 2024.
```

You can also use the modern `$( )` syntax:

```
echo "Today is $(date)"
```

13.1.3. What Gets Interpreted Inside Double Quotes?

- **Variable Expansion:** "My home is \$HOME"
- **Command Substitution:** "Time: \$(date)"

But wildcards stay literal:

```
echo "*.txt"
```

Output:

```
*.txt
```

Compare that with unquoted output:

```
echo *.txt
```

Output:

```
data.txt hello.txt new.txt
```

13.1.4. Breaking Long Lines

```
echo "This is a long string that \  
spans multiple \  
lines"
```

Output:

```
This is a long string that spans multiple lines
```

13.2. Using Single Quotes

Single quotes '...' completely disable interpretation. Everything inside is printed **exactly** as it appears.

13.2.1. Printing a Variable Name

```
echo '$USER'
```

Output:

```
$USER
```

No expansion happens. Bash treats everything inside as literal text.

13.2.2. Disabling Wildcards and Substitution

```
echo '*.txt'  
echo 'Today is $(date)'
```

Output:

```
*.txt  
Today is $(date)
```

13.2.3. When to Use Single Quotes

Use them when you **don't want** Bash to interpret anything:

- To print wildcards like *
- To show variable names like \$HOME
- To include command syntax like \$(...)

With single quotes, there is **no** expansion or evaluation.

13.3. Using Backslashes

The backslash \ escapes a single character, telling Bash to treat it literally—even inside unquoted or double-quoted strings.

13.3.1. Escaping Dollar Signs

```
echo \$PATH
```

Output:

```
$PATH
```

13.3.2. Escaping Quotes

```
echo \"Hello, Linux folks\"
```

Output:

```
"Hello, Linux folks"
```

This is equivalent to quoting the entire string:

```
echo "Hello, Linux folks"
```

13.3.3. When to Use Backslashes

Use them to:

- Escape special characters: `\$, *, \``
- Continue long lines: `\`
- Escape spaces in filenames
- Prevent globbing (wildcard expansion)

NOTES

Quoting gives you fine-grained control over how Bash treats special characters. Here's the difference:

- Use **double quotes** when you want to allow variable and command expansion but suppress globbing.
- Use **single quotes** when you want the entire string treated literally.
- Use **backslashes** to escape just one character at a time.

Once you understand these rules, you'll avoid surprises in your scripts—and prevent Bash from doing something unexpected with your special characters.

Chapter 14. Globbing in Bash

Globbing is Bash's built-in pattern-matching mechanism for filenames. It allows you to use wildcard characters to match multiple files or directories without having to name them one by one. While it may look similar to regular expressions, globbing is strictly used for filename expansion in the shell — not for pattern matching in tools like `grep` or `sed`.

14.1. What Is Globbing?

Bash expands wildcard patterns into matching filenames. This expansion happens **before** the command is run. The most commonly used globbing characters are:

- `*` – matches **zero or more** characters
- `?` – matches **exactly one** character
- `[ ]` – matches **one character** from a set or range

These patterns help automate file processing tasks in shell scripts or interactive use.

14.2. Matching Any String with `*`

The asterisk (`*`) matches any number of characters, including none.

List all files in the current directory

```
ls *
```

Match all `.txt` files

```
ls *.txt
```

This will match:

```
notes.txt  
report.txt  
summary.txt
```

But not:

```
image.png
```

14.3. Matching a Single Character with ?

The question mark (?) matches exactly **one** character. It's useful when you expect consistent filename structures.

Match files like `file1.txt`, `fileA.txt`

```
ls file?.txt
```

This would match:

```
file1.txt  
fileA.txt
```

but not:

```
file12.txt  
file.txt
```

14.4. Matching Specific Characters with []

Square brackets allow you to define a set or range of characters. Bash will match **any one character** from the list inside the brackets.

Match `file1.txt` to `file5.txt`

```
ls file[1-5].txt
```

Match any letter after "file", regardless of case

```
ls file[a-zA-Z]*.txt
```

This pattern matches:

```
filea.txt  
fileZebra.txt
```

14.5. Matching Hidden Files

In Unix-based systems, files beginning with a dot (.) are considered hidden. You can match them specifically using patterns that start with a dot.

List all hidden files and directories

```
ls -a
```

Match all hidden files with globbing

```
ls .*
```

This will include:

```
.bashrc  
.gitconfig  
.profile
```



The glob `does not` match hidden files by default. You must explicitly use `.` to include them.

Chapter 15. Pipelines and Redirections

15.1. Pipelines in Bash

Pipelines are one of Bash's most powerful features, allowing you to connect commands together so that the output of one becomes the input of the next. This chaining mechanism enables complex data processing to be written in a clean and efficient way.

15.1.1. What Is a Pipeline?

A pipeline is a sequence of commands connected by the pipe symbol (`|`). Bash sends the standard output (stdout) of the command on the left side of the pipe as standard input (stdin) to the command on the right.

Syntax

```
command1 | command2 | command3
```

Each command processes the data it receives and passes it along the chain. This allows for complex processing in one line.

15.1.2. Basic Example

Count the number of lines containing the word "error" in a log file

```
grep "error" log.txt | wc -l
```

Here's what happens:

- `grep "error" log.txt` finds all lines with the word "error".
- `wc -l` counts how many lines `grep` returned.

15.1.3. Filtering and Transforming Data

Pipelines are especially powerful when combined with text processing tools like `grep`, `sed`, `awk`, `cut`, and `sort`.

Replace the word "failure" with "success" in lines that contain "error"

```
grep "error" log.txt | sed 's/failure/success/g'
```

What's happening here:

- `grep` filters lines containing "error".
- `sed` replaces every instance of "failure" with "success" in those lines.

15.1.4. Combining Pipelines with Redirection

You can combine pipelines with redirection operators to save or read from files.

Save the output of a pipeline to a file

```
command1 | command2 > output.txt
```

Feed input to a pipeline from a file

```
command1 < input.txt | command2
```

You can also redirect standard error:

Redirect stderr to a file while piping stdout

```
command1 2> errors.txt | command2
```

15.1.5. A quick practical example

Extract and deduplicate log information

```
grep "error" log.txt | awk '{print $3, $5}' | sort | uniq > unique_errors.txt
```

Step-by-step breakdown:

- `grep "error" log.txt` filters lines with "error".
- `awk '{print $3, $5}'` selects the 3rd and 5th fields from each line.
- `sort` organizes the lines alphabetically.
- `uniq` removes duplicates.
- `>` writes the result to `unique_errors.txt`.

Pipelines let you:

- Combine multiple commands for cleaner, more expressive shell scripts.
- Pass data from one tool to another seamlessly.
- Process and transform large datasets without temporary files.

Use them whenever you find yourself doing multiple steps on the same data — they're the glue that makes shell scripting flexible and composable.

15.2. Redirections in Bash

When working with input and output in Bash, everything is treated as a stream of data. These streams are managed using **file descriptors**. By default, three file descriptors are always open in a shell session:

File Descriptor	Abbreviation	Description
0	STDIN	Standard input. This is usually your keyboard. When you use the input redirection symbol (<), Bash reads input from a file as if it were typed at the keyboard.
1	STDOUT	Standard output. This is usually your terminal screen. When you use the output redirection symbol (>), Bash writes the output to a file instead of the screen.
2	STDERR	Standard error. Like STDOUT, this normally writes to your screen, but is used specifically for error messages. You can redirect it using 2> to capture error output separately.



Unless explicitly closed, file descriptors always point to something—typically your terminal when Bash starts.

In addition to 0, 1, and 2, you can open and close custom file descriptors (3, 4, 5, ...), and even duplicate them to point to the same location. These are powerful tools for advanced scripting and redirection techniques.

Coming up next: how to use redirection symbols (>, >>, <, 2>, &>, and so on) in practice.

15.2.1. Redirection Operators

Redirection in Bash allows you to control where input comes from and where output goes. Below is a comprehensive list of redirection operators and what they do.

Redirection	Description
<code>command > file</code>	Redirect the standard output (stdout) of a command to a specified file. Same as <code>command 1> file</code> .
<code>command 2> file</code>	Redirect the standard error (stderr) of a command to a specified file.
<code>command >> file</code>	Append stdout of a command to a specified file without overriding existing data.
<code>command 2>> file</code>	Append stderr of a command to a specified file without overriding existing data.

Redirection	Description
<code>command &> file</code>	Redirect both stdout and stderr of a command to a file. Same as <code>command > file 2>&1</code> , but not <code>command 2>&1 > file</code> .
<code>command > /dev/null</code>	Discard stdout of a command. <code>/dev/null</code> discards all data written to it.
<code>command 2> /dev/null</code>	Discard stderr of a command.
<code>command &> /dev/null</code>	Discard both stdout and stderr of a command. Same as <code>command > /dev/null 2>&1</code> .
<code>command < file</code>	Redirect the contents of a file to stdin of a command. Same as <code>command 0< file</code> .
<code>command << EOL</code>	Redirect multiple lines to stdin using a here-document. Ends when a line with only EOL is found.
<code>command <<- EOL</code>	Same as <code><<</code> but strips leading tab characters from the input.
<code>command <<< 'string'</code>	Use a here-string to redirect input from a single string.
<code>command <<< \$word</code>	Same as above but using a variable.
<code>command > file</code>	Overrides the <code>noclobber</code> shell option to force file overwrite.
<code>exec 2> file</code>	Redirect stderr of all commands to a file.
<code>exec > file</code>	Redirect stdout of all commands to a file.
<code>exec 3< file</code>	Create custom file descriptor (3) for reading from a file.
<code>exec 3> file</code>	Create custom file descriptor (3) for writing to a file.
<code>exec 3<> file</code>	Create custom file descriptor (3) for both reading and writing.
<code>exec 3>&-</code>	Close the created file descriptor.
<code>exec 4<&3</code>	Make file descriptor 4 a copy of file descriptor 3.
<code>exec 4>&3-</code>	Copy file descriptor 3 to 4 and close file descriptor 3.
<code>echo "foo" >&3</code>	Write to custom file descriptor 3.
<code>cat <&3</code>	Read from custom file descriptor 3.

Redirection	Description
<code>exec 3<>/dev/tcp/host/port</code>	Open a TCP connection. Bash feature only.
<code>exec 3<>/dev/udp/host/port</code>	Open a UDP connection. Bash feature only.
<code>(command1; command2) > file</code>	Redirect stdout from multiple commands to a file using a subshell.
<code>{ command1; command2; } > file</code>	Redirect stdout from multiple commands to a file without subshell. Faster.
<code>command < <(command1)</code>	Redirect stdout of <code>command1</code> to an anonymous pipe and pass to <code>command</code> .
<code>command < <(command1)</code>	Same as above. Example: <code>diff <(find /path1 sort) <(find /path2 sort)</code>
<code>command < <(command1) < <(command2)</code>	Redirect output of two commands to anonymous pipes and pass to another command.
<code>command1 >(command2)</code>	Run <code>command2</code> with stdin connected to a named pipe and pass filename to <code>command1</code> .
<code>command1 > >(command2)</code>	Run <code>command2</code> with stdout from <code>command1</code> through a named pipe.
<code>command1 command2</code>	Redirect stdout of <code>command1</code> to stdin of <code>command2</code> .
<code>command1 & command2</code>	Redirect stdout and stderr of <code>command1</code> to stdin of <code>command2</code> . (Bash 4.0+)
<code>command tee file</code>	Redirect stdout of <code>command</code> to both a file and screen.
<code>exec {filew}> file</code>	Open file for writing using a named file descriptor (<code>filew</code>). Bash 4.1+.
<code>command 3>&1 1>&2 2>&3</code>	Swap stdout and stderr of a command.

Redirection	Description
<code>cmd > >(cmd1) 2> >(cmd2)</code>	Send stdout to cmd1 and stderr to cmd2.
<code>cmd1 cmd2 cmd3 cmd4; echo \\${PIPESTATUS[@]}</code>	Show exit codes of all piped commands.

Chapter 16. Exit Status and the `exit` Command

In Bash, every command you run has an exit status. This exit status is a numeric code that tells you whether the command succeeded or failed. Understanding and using exit codes is essential for scripting logic and error handling.

16.1. What Is an Exit Status?

Every command that runs in Bash returns an exit status when it completes:

- `0` means success — the command ran without any errors.
- Any non-zero number indicates a failure or an error.

Bash provides a special variable, `$?`, which always holds the exit status of the last executed command.

Successful command

```
cat manifesto.txt  
echo $?
```

Output

```
A Gentle reminder, I use Arch BTW!  
0
```

In this example, `cat` successfully displayed the file, so the exit status is `0`.

Failed command

```
cat no-file  
echo $?
```

Output

```
cat: no-file: No such file or directory  
1
```

This time, the file doesn't exist, so the command fails and the exit code is `1`.

The exact exit codes and their meanings can vary depending on the command. You can often find exit code definitions in the command's `man` page.

16.2. Exit Status with Pipelines

When using pipes (|) to chain commands, the exit status stored in `$?` will be the exit code of the **last command** in the pipeline — not the first, even if the first one failed.

Last command determines the exit status

```
ls -lah | head -n 4
echo $?
```

Output

```
total 88K
drwxr-x--- 5 mint mint 4.0K Mar 13 22:24 .
drwxr-xr-x 5 root root 4.0K Nov 22 13:54 ..
-rw----- 1 mint mint 5.6K Mar  5 17:20 .bash_history
0
```

The head command ran fine, so the exit status is 0, even if ls had failed, it wouldn't show up here.

First command fails, but exit status is still 0

```
ls -lah no-directory | head -n 4
echo $?
```

Output

```
ls: cannot access 'no-directory': No such file or directory
0
```



Even though `ls` failed, the exit status reflects the result of `head`, which succeeded. This behavior is important to remember when checking for command failures inside a pipeline.

To retrieve the exit status of **all** commands in a pipeline, use the `${PIPESTATUS[@]}` array:

```
ls no-dir | head
echo "${PIPESTATUS[@]}"
```

16.3. The `exit` Command

The `exit` command ends a Bash script and returns an exit status to the shell.

```
exit N
```

- If `N` is provided, `exit` returns that as the script's exit code.
- If no value is provided, the script exits with the status of the last command.



This also applies to the interactive shell. Typing `exit 0` will close your shell session with a "successful" status.

16.4. Using `exit` in Scripts

You can control the exit status of a script to communicate success or failure to whatever called it. This is useful for automation tools, cron jobs, and error handling logic.

```
#!/bin/bash

FILE=~/.logs.txt

if test -f "$FILE"; then
    echo "$FILE exists."
    exit 2
else
    echo "$FILE doesn't exist."
    exit 4
fi
```

- If the file exists, the script exits with status 2.
- If it doesn't, it exits with 4.

You can check this afterward using:

```
echo $?
```

16.5. What If You Don't Pass an Argument?

If no argument is passed to `exit`, it returns the exit code of the **last executed command**:

```
#!/bin/bash

FILE=~/.logs.txt

if test -f "$FILE"; then
```

```
    echo "$FILE exists."
    exit
else
    echo "$FILE doesn't exist."
    touch logs.txt
    exit
fi
```

In this case:

- If the file exists, the exit code will match the exit status of the `echo` command.
- If not, the exit status will match that of the `touch` command.



If you don't use `exit` explicitly at all, the script still exits using the last command's status.

Chapter 17. Bash History

Every time you enter a command in the Bash shell, it gets recorded in a history file. This allows you to recall, repeat, and modify previous commands—saving time and keystrokes.

17.1. Viewing and Managing Command History

The history built-in command is your main interface to view and control past commands.

Command	Description
<code>history</code>	Display the full list of previously executed commands.
<code>history n</code>	Show only the last <code>n</code> commands from the history list.
<code>history -c</code>	Clear your current session history. New commands will still be added afterward.
<code>history -d n</code>	Delete the <code>n</code> th entry from the history list.
<code>sudo !!</code>	Re-run the last command with <code>sudo</code> . Extremely handy when you forget privileges.
<code><space> command</code>	Avoid saving a command to history by prefixing it with a space (if <code>HISTCONTROL=ignorespace</code> is set).
<code>shopt -s histverify</code>	Prevent immediately running a command after history expansion; allows review before execution.
<code>echo \$HISTSIZE</code>	Print the number of commands Bash stores in memory for the current session.



The default history size is typically 500 or 1000 commands, but you can change this by modifying `HISTSIZE` in your shell configuration file.

17.1.1. History Event Designators

Event designators let you refer to past commands using `!` shortcuts.

Event	Description
<code>!!</code>	Expand to the previous command.
<code>!-n</code>	Expand to the <code>n</code> th most recent command (counting backwards).
<code>!n</code>	Expand to the command with index <code>n</code> in the history list.

Event	Description
! <code><command></code>	Expand to the most recent use of <code><command></code> .
! <code><text></code>	Expand to the last command that begins with <code><text></code> .
! <code>n:p</code> or <code>!<code><text></code>:p</code>	Print the expanded command, but don't execute it.
<code>old</code> <code>new</code>	Re-run the last command, replacing <code>old</code> with <code>new</code> .
! <code>#</code>	Expand to the entire command line typed so far.



History expansion is powerful but can be dangerous. Use `:p` to preview before running if unsure.

17.2. Word Designators

Word designators let you target parts of a previous command (its arguments or parameters).



You append a word designator after the event using a colon (`:`). For example: `!!: $` expands to the **last** argument of the previous command.

Word	Description
! <code>\$</code>	Expand the last argument of the previous command.
! <code>*</code>	Expand all arguments of the previous command.
! <code>^</code>	Expand the first argument of the previous command.
! <code>%</code>	Expand the word matched in the most recent <code>?string?</code> search.
! <code>x-y</code>	Expand a range of arguments from position <code>x</code> to <code>y</code> .

17.3. Modifiers for Word Designators

After selecting a word or range, you can apply **modifiers** to manipulate it:

- `:h` → Remove trailing filename, leaving directory
- `:t` → Remove leading path, leaving filename
- `:r` → Remove file extension
- `:e` → Extract file extension
- `:p` → Print the command after expansion, don't execute

Strip filename and print

```
$ cat demo.sh
$ !$:h:p
```

Output

```
demo
```



Modifiers can be chained and are useful for quick one-liners when manipulating file paths from history.

17.3.1. History Modifiers

Once you've referenced a previous command using an event designator like `!!` or `!$`, you can apply **modifiers** to tweak parts of it before execution. These modifiers help transform file paths, substitute text, and control execution.

Modifier	Description
<code>:h</code>	Strip the file name , leaving the directory path (head). Think of it as trimming everything after the last <code>/</code> .
<code>:t</code>	Strip the directory path , leaving only the file name (tail).
<code>:r</code>	Remove the file extension (trailing <code>.ext</code>) from a filename, keeping the root name.
<code>:e</code>	Keep only the file extension (trailing <code>.ext</code>), stripping everything else.
<code>:p</code>	Print the final command after substitution, but don't run it. Great for previewing before execution.
<code>:q</code>	Quote the substituted words to prevent further expansion or splitting.
<code>:x</code>	Like <code>:q</code> , but also breaks the string into words at spaces and newlines, quoting each part individually.
<code>:s/old/new/</code>	Replace the first occurrence of <code>old</code> with <code>new</code> .
<code>:gs/old/new/</code>	Replace all occurrences of <code>old</code> with <code>new</code> . The <code>g</code> means global.
<code>:&</code>	Repeat the last substitution done with <code>:s</code> .
<code>:G</code>	Apply the <code>:s</code> or <code>:&</code> modifier once per word in the event, rather than to the full line.



You can chain modifiers together, like `!!:h:p` to strip the filename and preview the command.

17.3.2. History Slices

You can extract **specific arguments** (called tokens) from a previous command using slices. This is helpful when you only want part of the last command.

Each word (or token) in a command is numbered, starting at zero for the command itself.

Slice	Description
!!:n	Expand only the nth word (0 is the command, 1 is the first argument, and so on).
!^	Expand the first argument from the most recent command. Equivalent to !!:1.
!\$	Expand the last argument from the most recent command.
!!:n-m	Expand a range of words from position n to m.
!!:n-\$	Expand from word n to the last word .



Slices and modifiers work together. For example, !!:1:r expands the first argument of the previous command and removes its file extension.

Chapter 18. Bash Options

Bash options are configuration settings that modify the behavior of the shell or a script. You can enable or disable these options using the `set` builtin.

There are two main syntaxes to control options:

- `set -o option-name` or `set -option-abbrev` to **enable** an option.
- `set +o option-name` or `set +option-abbrev` to **disable** an option.

To enable an option **only when running a script**, you can do:

```
bash -o option-name script.sh
# or
bash -option-abbrev script.sh
```



Options affect how scripts behave and are critical when building reliable, secure, and predictable shell scripts.

18.1. Common Bash Options

Abbreviation	Name	Effect
<code>-B</code>	brace expansion	Enable brace expansion (e.g., <code>{a,b,c}</code> becomes <code>a b c</code>). This is on by default.
<code>+B</code>	brace expansion	Disable brace expansion.
<code>-C</code>	noclobber	Prevent overwriting files via output redirection. You can still override with <code>> </code> .
<code>-D</code>	—	Print double-quoted strings prefixed with <code>\$</code> but do not execute commands. Used for debugging.
<code>-a</code>	allexport	Automatically export all variables as environment variables.
<code>-b</code>	notify	Notify when background jobs complete (not typically useful in scripts).
<code>-c '...'</code>	—	Read and execute commands from the given string instead of a script or stdin.

Abbreviation	Name	Effect
—	checkjobs	Warns about unfinished background jobs on shell exit. Enable with <code>shopt -s checkjobs</code> . Use with caution—it may hang!
-e	errexit	Exit the script if any command fails (non-zero exit status). Critical for robust scripting.
-f	noglob	Disable filename expansion (globbing) using wildcards like <code>*</code> and <code>?</code> .
—	globstar	Enable for recursive globbing (e.g., <code>/*.txt</code>). Enable with <code>shopt -s globstar</code> .
-i	interactive	Run the script in interactive mode (useful for REPL-like input).
-n	noexec	Don't execute any commands—just check for syntax errors.
-o name	—	Enable the named shell option (like <code>pipefail</code> or <code>posix</code>).
-o posix	POSIX mode	Make Bash behave like a POSIX-compliant shell. Useful for portability.
-o pipefail	pipe failure	Make a pipeline return the exit code of the first failed command instead of the last one.
-p	privileged	Run the script with elevated privileges. Be very cautious with this!
-r	restricted	Enable restricted mode , limiting what the shell can do (e.g., no <code>cd</code> or <code>exec</code>).
-s	stdin	Read script input from stdin instead of a file.
-t	onecmd	Run just one command and exit.
-u	nounset	Treat undefined variables as errors and abort the script. Great for catching typos.
-v	verbose	Print each command to stdout before execution.
-x	xtrace	Print commands and their expanded arguments (useful for debugging).
-	—	End of options. Everything after this is treated as a positional parameter.

Abbreviation	Name	Effect
--	—	Unset all positional parameters. If arguments are given (-- arg1 arg2), positional parameters are set to them.

You can combine multiple options in one go:

```
set -eux # Exit on error, treat unset variables as errors, print commands
```

18.2. Additional Bash Options

Bash includes extra optional behaviors that aren't controlled with `set`, but with another builtin: `shopt`. This command lets you enable or disable various shell behaviors that are especially useful in scripting and interactive use.

18.2.1. The `shopt` Builtin

Use `shopt` to view or modify these shell options.

```
shopt [-pqsu] [-o] <OPTION...>
```



- Use `-s` to **set** an option
- Use `-u` to **unset** an option
- Use `-p` to **print** all shell options and their status (default)
- Use `-q` to **query** quietly (exit status only, no output)
- Use `-o` to restrict to options also used by `set`



If you run `shopt -s nullglob`, then glob patterns that don't match anything will expand to nothing instead of staying as-is. This is extremely helpful to avoid errors in loops.

18.2.2. Exit Code Behavior

When you list or change options using `shopt`, the command sets an appropriate exit status:

- **0 (true)** if the operation was successful (or all options are enabled when queried).
- **1 (false)** if one or more options are disabled or an unknown option was referenced.

Example: Enable nullglob

```
shopt -s nullglob
```

18.2.3. shopt Options Summary

Flag	Meaning
-s	Set (enable) the specified option(s)
-u	Unset (disable) the specified option(s)
-p	Print all shell options and their status (default)
-q	Quiet mode. Exit status reflects whether all named options are enabled
-o	Restrict options to those also accepted by set

18.3. Additional Shell Options

These are some of the most useful options you can enable with `shopt -s` or disable with `shopt -u`:

Option	Description
autocd	Allows you to type a directory name directly to <code>cd</code> into it, without using the <code>cd</code> command.
cdspell	Automatically corrects small typos in <code>cd</code> directory names (e.g. transposed characters, missing/extra letters).
checkjobs	Prevents the shell from exiting if there are running or stopped jobs. You must exit twice to leave.
dirspell	Enables spelling correction for directory names used in glob patterns.
dotglob	Makes glob patterns like <code>*</code> also match hidden files (those beginning with a dot).
nullglob	Causes globs that don't match any files to expand to nothing instead of remaining as-is.
failglob	Makes Bash throw an error if a glob pattern fails to match any files.
nocaseglob	Makes glob matching case-insensitive.
globstar	Enables the <code>**</code> pattern to recursively match files and directories.

To enable multiple options at once, you can list them:

```
shopt -s dotglob nullglob globstar
```

For a full list of shopt options, check the [official Bash manual](#).

Chapter 19. Regular Expressions in Bash

Regular expressions (regex) are patterns used to match and manipulate strings. They're an essential tool when working with commands like `grep`, `sed`, and `awk`, and are commonly used in shell scripts for filtering and parsing text.

19.1. Types of Regular Expressions

There are two main types of regex syntax you'll encounter in Bash:

- **POSIX Basic Regular Expressions (BRE)** – used by default in tools like `grep`.
- **POSIX Extended Regular Expressions (ERE)** – used with tools like `egrep` or `grep -E`, and `awk`. These allow for more complex patterns without the need for excessive escaping.



For most use cases in scripting or on the command line, POSIX EREs are more readable and flexible.

19.2. Anchors

Anchors are special characters that match positions rather than actual characters.

Pattern	Description
<code>^</code>	Matches the start of a string or start of a line in multiline mode
<code>\A</code>	Matches only the start of a string
<code>\$</code>	Matches the end of a string or end of a line in multiline mode
<code>\Z</code>	Matches only the end of a string
<code>\b</code>	Matches a word boundary (between word and non-word characters)
<code>\B</code>	Matches a position that is not a word boundary
<code>\<</code>	Matches the beginning of a word (GNU extension)
<code>\></code>	Matches the end of a word (GNU extension)

19.3. Character Classes

Character classes allow you to define a set or range of characters that a position in your regex can match.

Pattern	Description
<code>[abc]</code>	Matches any one character: a, b, or c

Pattern	Description
[^abc]	Matches any character except a, b, or c (the caret ^ negates character classes)
[a-z]	Matches any lowercase letter from a to z
[^a-z]	Matches any character except lowercase letters a to z
[0-9]	Matches any digit from 0 to 9
[a-zA-Z]	Matches any letter (uppercase or lowercase)
[a-zA-Z0-9]	Matches any alphanumeric character



You can combine ranges and specific characters inside a single class: [a-cXYZ0-9]

19.4. Shorthand Character Classes

Many regex engines (especially Perl-compatible ones) support shorthand symbols for common character classes.

Pattern	Description
\c	Matches a control character
\s	Matches any whitespace (space, tab, newline)
\S	Matches any non-whitespace character
\d	Matches any digit (equivalent to [0-9])
\D	Matches any non-digit character
\w	Matches any "word" character: [a-zA-Z0-9_]
\W	Matches any non-word character
\x	Matches a hexadecimal digit (implementation-specific)
\0	Matches an octal digit (implementation-specific)



The behavior of \x and \0 can vary across tools. Use [0-7] or [0-9a-fA-F] when you want to be explicit.

19.5. POSIX Character Classes (for BRE/ERE)

POSIX character classes make your regular expressions more portable and readable by replacing common character

ranges with symbolic names. These are especially useful inside [and] brackets when working with tools like grep, sed, and awk.

Class	Equivalent	Description
<code>[[:alnum:]]</code>	<code>[0-9A-Za-z]</code>	Matches any alphanumeric character 0–9, A–Z, or a–z
<code>[[:alpha:]]</code>	<code>[A-Za-z]</code>	Matches any alphabetical character, either upper or lower case
<code>[[:blank:]]</code>	<code>[\t]</code>	Matches a space or Tab character
<code>[[:digit:]]</code>	<code>[0-9]</code>	Matches a numerical digit from 0 through 9
<code>[[:graph:]]</code>	<code>[[:alnum:]][[:punct:]]</code>	Visible characters (not space)
<code>[[:lower:]]</code>	<code>[a-z]</code>	Matches any lowercase alphabetical character a–z
<code>[[:print:]]</code>	<code>[-~] == [[:graph:]]</code>	Matches any printable character
<code>[[:punct:]]</code>	<code>[!\"#\$%&'()*+, \- ./:; \<@[\]^_{ }~]+'`</code>	Matches a punctuation character
<code>[[:space:]]</code>	<code>[\t\n\v\f\r]</code>	Matches any whitespace character: space, Tab, NL (newline), FF (formfeed), VT (vertical tab), CR (carriage return)
<code>[[:upper:]]</code>	<code>[A-Z]</code>	Matches any uppercase alphabetical character A–Z
<code>[[:word:]]</code>	<code>[0-9A-Za-z_]</code>	Matches all word characters
<code>[[:xdigit:]]</code>	<code>[0-9A-Fa-f]</code>	Matches all hexadecimal digits
<code>[[:<:]]</code>	<code>[\b(?:\w)]</code>	Matches start of word
<code>[[:>:]]</code>	<code>[\b(?:\w)]</code>	Matches end of word
<code>[[:ascii:]]</code>	<code>[\x00-\x7F]</code>	ASCII codes 0–127
<code>[[:cntrl:]]</code>	<code>[\x00-\x1F\x7F]</code>	Control characters



POSIX classes should only be used inside bracket expressions, like `[:digit:]`.

19.6. Quantifiers

Quantifiers let you specify how many times a part of your regex should match. They are essential for writing flexible and powerful patterns.

Pattern	Description
<code>?</code>	Matches the preceding element zero or one time
<code>*</code>	Matches the preceding element zero or more times
<code>+</code>	Matches the preceding element one or more times
<code>[0-9]+</code>	Matches one or more digits (0 through 9)
<code>{m}</code>	Matches the preceding element exactly <i>m</i> times
<code>{m,n}</code>	Matches the preceding element at least <i>m</i> times, but no more than <i>n</i> times
<code>{m,}</code>	Matches the preceding element <i>m</i> or more times
<code>a*</code>	Greedy: matches as many occurrences of <i>a</i> as possible
<code>a*?</code>	Lazy: matches as few as possible while still allowing the overall pattern to match
<code>a*+</code>	Possessive: matches as many as possible without backtracking
<code>{,n}</code>	Matches the preceding element up to <i>n</i> times

19.7. Escaping Special Characters

Some characters have special meanings in regular expressions. To match them literally, you must prefix them with a backslash (`\`).

Common Meta-characters

```
^ ( { | * < $ ) ? > .  
[ \
```

Escape these special characters with `\`



When in doubt, escape characters using `\` to avoid unintended matches.

19.8. Groups and Ranges

Regular expressions support grouping, alternation, and ranges to match more complex patterns. These features are particularly useful when you're trying to match multiple possibilities in one go.

Pattern	Description
<code>.</code>	Matches any character except a newline (<code>\\n</code>)
<code>(a b)</code>	Matches either a or b (alternation)
<code>(...)</code>	Capturing group: groups part of the regex and remembers the match
<code>(?:...)</code>	Non-capturing group: groups part of the regex without capturing it
<code>[abc]</code>	Matches any single character: a, b, or c
<code>[^abc]</code>	Matches any character except a, b, or c
<code>[a-q]</code>	Matches any lowercase letter from a to q
<code>[A-Q]</code>	Matches any uppercase letter from A to Q
<code>[0-7]</code>	Matches any digit from 0 to 7
<code>\x</code>	Refers to a previously matched sub-pattern (by number)

19.9. Pattern Modifiers

These modifiers tweak how the regex engine behaves when interpreting the pattern.

Modifier	Description
<code>g</code>	Global match: apply the regex to all matches in the input, not just the first
<code>i</code>	Case-insensitive matching
<code>m</code>	Treat input as multiple lines (affects <code>^</code> and <code>\$</code> anchors)
<code>s</code>	Treat input as a single line (dot <code>.</code> will match newline too)
<code>x</code>	Ignore whitespace and allow comments in pattern (for readability)
<code>e</code>	Evaluate replacement as code (used in some <code>sed</code> variants)
<code>U</code>	Makes quantifiers ungreedy by default

19.10. Assertions

Assertions are zero-width conditions—meaning they don’t consume characters but still affect matches. Think of them as invisible checks.

Assertion	Description
(?=...)	Positive lookahead: checks for a match ahead without consuming it
(?!...)	Negative lookahead: ensures a pattern is not ahead
(?<=...)	Positive lookbehind: checks for a pattern behind the current position
(?<!...)	Negative lookbehind: ensures a pattern doesn’t exist behind
(?>...)	Atomic group: once matched, prevents backtracking
(?(cond)yes)	Conditional match (rare)
(?(cond)yes\ no)	Conditional with else clause
(?# comment)	Inline comment (only supported in some engines)

19.11. String Replacement Variables

Used in tools like sed, Perl, and some bash string substitutions, these refer to portions of the original match.

Symbol	Description
\$n	Contents of the nth capture group
\$2	Second capture group (e.g. xyz in /^(abc(xyz))\$/)
\$1	First capture group (e.g. xyz in /^(?:abc)(xyz)\$/)
\$`	Part of the string before the match
\$'	Part of the string after the match
\$+	Last matched group
\$&	Entire matched string

19.12. Escape Sequences

Some characters must be escaped to be interpreted literally or to represent invisible characters.

Sequence	Description
\\	Escape the next character
\Q	Begin literal mode (treat all characters literally)
\E	End literal mode (resume special character interpretation)



Escaping is how you prevent special characters from acting like metacharacters. For example, to match a literal dot (.), use \. instead of ..

19.13. Special Characters

These sequences represent common invisible characters or encodings.

Sequence	Description
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\f	Form feed
\xxx	Octal value for character
\xhh	Hexadecimal value for character

19.14. Search for Lines That Start with a Word

```
grep '^ERROR' /var/log/syslog
```

This command finds all lines in `/var/log/syslog` that start with the word `ERROR`.

- `^` is the regex anchor for the beginning of the line.
- This ensures it doesn't match lines where `ERROR` appears later in the line.



Use quotes around patterns with special characters like `^` to prevent shell expansion.

19.15. Match IP Addresses Using **grep**

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log
```

- -E enables extended regex.
- -o prints only the matched portion, not the whole line.
- ([0-9]{1,3}\.){3}[0-9]{1,3} matches typical IPv4 addresses.



This matches numbers like 999.999.999.999, which aren't valid IPs—but it's fine for quick parsing. Use stricter patterns for validation.

19.16. Find Email Addresses in a Text File

```
grep -Eo '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}' newsletter.txt
```

This matches most standard email formats.

- + matches one or more of the preceding character set.
- @ is literal.
- [A-Za-z]{2,} ensures domain extensions are at least 2 letters long.

19.17. Replace Tabs with Commas Using **sed**

```
sed 's/\t/,/g' input.tsv > output.csv
```

- \t matches tab characters.
- s/old/new/g is the basic sed substitution.
- This converts tab-separated files to comma-separated format.

19.18. Print Lines That Don't Contain a Pattern

```
grep -vE 'DEBUG|TRACE' app.log
```

- -v inverts the match.
- -E allows extended regex, so you can use | to match multiple patterns.
- This filters out noisy log levels.

19.19. Extract Usernames from /etc/passwd

```
awk -F: '{ print $1 }' /etc/passwd | grep '^[a-z]'
```

- `awk -F:` sets the delimiter to `:`.
- `{ print $1 }` grabs the username field.
- `grep '^[a-z]'` filters for lowercase usernames only.

Appendix A: Terminal Shortcuts Reference

A.1. Bash Navigation

Shortcut	Action
Ctrl + A	Move to the start of the command line
Ctrl + E	Move to the end of the command line
Ctrl + F	Move one character forward
Ctrl + B	Move one character backward
Ctrl + XX	Switch cursor between start of line and current position
Ctrl +] + x	Move forward to next occurrence of x
Alt + F / Esc + F	Move one word forward
Alt + B / Esc + B	Move one word backward
Alt + Ctrl +] + x	Move to previous occurrence of x

A.2. Bash Control and Process Handling

Shortcut	Action
Ctrl + L	Clear the terminal screen
Ctrl + S	Pause command output to the screen
Ctrl + Q	Resume paused output
Ctrl + Z	Suspend current command and move to background
Ctrl + C	Send SIGINT to terminate current command
Ctrl + D	Close the current terminal or send EOF

A.3. Bash History Shortcuts

Shortcut	Action
Ctrl + R	Incremental reverse search through history
Alt + P	Non-incremental reverse search

Shortcut	Action
Ctrl + J	End history search at current result
Ctrl + _	Undo last command change
Ctrl + P / Up	Previous command
Ctrl + N / Down	Next command
Ctrl + S	Forward search in history (may need to disable terminal flow control)
Ctrl + O	Execute and recall command from search
Ctrl + G	Exit history search

A.4. Bash History Expansions

Shortcut	Action
!!	Repeat the last command
!*	Repeat all arguments from last command
!*:p	Print expanded arguments
!x	Run last command starting with x
!x:p	Print last command starting with x
!\$	Use last argument from previous command
!^	Use first argument from previous command
!\$:p	Print last argument
^{old} _{new}	Replace old with new in last command
!n:m	Use arguments n to m from command !n
!n	Run command number n
!n:p	Print command number n
!n:\$	Use arguments n to last from command n
!fi	Run last command starting with fi

A.5. Bash Line Editing

Shortcut	Action
Ctrl + U	Delete from cursor to beginning of line
Ctrl + K	Delete from cursor to end of line
Ctrl + W	Delete word before cursor
Ctrl + D	Delete character under cursor
Ctrl + H	Delete character before cursor
Alt + D	Delete from cursor to end of word
Alt + Backspace	Delete from cursor to start of word
Alt + . / Esc + .	Insert last argument from previous command
Alt + <	Move to beginning of history
Alt + >	Move to end of history
Esc + T	Swap last two words before cursor
Alt + T	Swap current and previous words

A.6. Bash Autocomplete and Information

Shortcut	Action
TAB	Autocomplete command or file
~TAB TAB	List all system users
Ctrl + I	Same as TAB (autocomplete)
Alt + ?	List files/folders in path for help
Alt + *	Insert all files/folders in path as argument

A.7. Bash Special Parameters

Special parameters are predefined variables set and used by the shell. Unlike regular variables, their names are usually a single symbol or a digit, and they serve specific purposes during script execution.

A.7.1. Positional Parameters

Parameter	Description
\$0	The name of the script or shell
\$1 to \$9	The first to ninth arguments passed to the script or function
\${10} and beyond	Access to arguments beyond the ninth; requires braces
\$#	The number of positional parameters
\$*	All positional parameters as a single word
\$@	All positional parameters as separate words

The difference between `$*` and `$@` is only visible when quoted:



- `"$"` expands to one string: `"arg1 arg2 arg3"`
- `"$@"` expands to multiple strings: `"arg1" "arg2" "arg3"`

A.7.2. Other Special Parameters

Parameter	Description
\$?	Exit status of the last command
\$\$	PID of the current shell
\$_	PID of the last background command
\$-	Current option flags set by <code>set</code>
\$_	The last argument to the previous command



Special parameters are readonly and managed by the shell itself—you don't assign to them; you use them to access internal shell state.

A.8. Bash Special Parameters

Bash provides a set of special parameters that behave differently from normal variables. They are typically a single symbol and are used to retrieve metadata about the current shell state, scripts, and arguments.

Here's a breakdown of the most common ones:

A.8.1. \$? — Exit Status of Last Command

This parameter returns the exit status of the last executed command.

```
ls /not-a-real-dir
echo $?
```

If the command fails, `$?` returns a non-zero value. If it succeeds, it returns `0`.



Always check `$?` immediately after a command. Running another command will overwrite its value.

A.8.2. `$0` — Name of the Script

When running a script, `$0` contains the name (or path) used to invoke the script.

```
echo "Script name is $0"
```

Useful for printing usage messages.

A.8.3. `$#` — Number of Positional Parameters

Returns the number of arguments passed to the script.

```
./myscript.sh arg1 arg2
# Inside the script
echo "You passed $# arguments"
```

A.8.4. `$@` and `$*` — All Positional Parameters

Both expand to all positional parameters, but behave differently when quoted.

- `$@` treats each argument as a separate word.
- `$*` treats the entire list as a single word.

```
for arg in "$@"; do
    echo "$arg"
done
```



Prefer `"$@"` in scripts to preserve the integrity of each argument.

A.8.5. `#!` — PID of Last Background Process

Returns the process ID of the most recently executed background command.

```
sleep 30 &
echo "PID: $!"
```

A.8.6. \$\$ — PID of Current Shell

This parameter expands to the process ID of the current shell.

```
echo "Current shell PID: $$"
```

A.8.7. \$- — Current Shell Options

Shows the current shell options (like `interactive`, `xtrace`, etc.).

```
echo $-
```

A.8.8. \$?, \$#, \$@, and more...

These are the core special parameters you'll frequently encounter. We'll cover more as we go deeper into scripting scenarios.

A.9. Bash Signals

Signals are a way for processes to receive notifications from the operating system or other processes. Each signal has a number and a name, and some signals can be handled or trapped by a script using the `trap` command.

A.9.1. Common Signals

Name	Number	Description
SIGHUP	1	Hangup detected on controlling terminal or death of controlling process
SIGINT	2	Interrupt from keyboard (Ctrl+C)
SIGQUIT	3	Quit from keyboard (Ctrl+\)
SIGILL	4	Illegal instruction
SIGABRT	6	Abort signal
SIGFPE	8	Floating point exception
SIGKILL	9	Kill signal (cannot be caught or ignored)

Name	Number	Description
SIGSEGV	11	Invalid memory reference (segmentation fault)
SIGPIPE	13	Broken pipe
SIGALRM	14	Timer signal
SIGTERM	15	Termination signal
SIGUSR1	10	User-defined signal 1
SIGUSR2	12	User-defined signal 2
SIGCHLD	17	Child process terminated or stopped
SIGCONT	18	Continue if stopped
SIGSTOP	19	Stop process (cannot be caught or ignored)
SIGTSTP	20	Stop typed at terminal (Ctrl+Z)
SIGTTIN	21	Background process attempting read
SIGTTOU	22	Background process attempting write

A.9.2. Viewing Available Signals

You can list all signal names and their numbers with the `kill` command:

```
kill -l
```

A.9.3. Sending Signals

To send a signal to a process, use:

```
kill -SIGTERM <PID>
kill -9 <PID>      # SIGKILL, force terminate
```

A.9.4. Handling Signals in Scripts

Use the `trap` command to catch and handle signals in your scripts:

```
trap 'echo "Interrupted! Cleaning up..."; exit 1' SIGINT
```



SIGKILL and **SIGSTOP** cannot be trapped or ignored.

A.10. Prompt Customization Examples

The appearance of your Bash prompt is controlled by special environment variables:

- PS1: The primary prompt string.
- PS2: The secondary prompt (used for multiline commands).
- PS3: The prompt used for `select` in scripts.
- PS4: The prompt used when executing with `set -x` (useful for debugging).

A.10.1. Basic PS1 Example

```
PS1="\u@\h:\w\$ "
```

This produces a prompt like:

```
user@host:/current/dir$
```

Legend:

- \u: Username
- \h: Hostname (short)
- \w: Current working directory
- \\$: Shows \$ (user) or # (root)

A.10.2. Add Color to the Prompt

You can add colors using ANSI escape codes. Here's an example with color:

```
PS1="\[\e[0;32m\]\u@\h\[\e[0m\]:\[\e[0;34m\]\w\[\e[0m\]\$ "
```

This makes the username and hostname green, and the directory blue.



The `\[` and `\]` are necessary to properly calculate prompt length when using color escape sequences.

A.10.3. Customize PS2

The default value of PS2 is `>` . It appears when Bash expects continuation of a multiline command.

You can change it like this:

```
PS2=". . .> "
```

A.10.4. Customize PS4

Useful when debugging scripts with `set -x`. Example:

```
export PS4='+ ${BASH_SOURCE}:${LINENO}:${FUNCNAME[0]}: '
set -x
```

This will prefix trace output with the script name, line number, and function name.

A.10.5. Prompt with Git Branch

You can add Git branch info to your prompt like this:

```
PS1='\u@\h:\w$(git branch 2>/dev/null | grep "^*" | sed "s/*/ (/;s$/)/")\$ '
```

This shows the current Git branch in parentheses when inside a Git repository.

For more advanced prompts, consider using frameworks like `starship`, or tools like `bash-git-prompt`.

A.11. Reserved Bash Keywords

Bash has a set of reserved words that cannot be used as variable names or function names. These keywords have special meaning in the shell syntax and are used for control flow, functions, and other structural elements.

Here's a reference of the most common Bash reserved words:

Keyword	Description
<code>if</code>	Begins a conditional block.
<code>then</code>	Marks the start of commands to execute if the condition is true.
<code>else</code>	Specifies commands to execute if the condition is false.
<code>elif</code>	Else-if: checks another condition if the first <code>if</code> fails.
<code>fi</code>	Ends the <code>if</code> block.

Keyword	Description
<code>case</code>	Begins a case statement for pattern matching.
<code>esac</code>	Ends a case block (reverse of <code>case</code>).
<code>for</code>	Begins a loop over a list.
<code>in</code>	Used with <code>for</code> to list items to iterate over.
<code>while</code>	Starts a loop that runs while a condition is true.
<code>until</code>	Starts a loop that runs until a condition becomes true.
<code>do</code>	Marks the start of the loop body.
<code>done</code>	Ends a <code>for</code> , <code>while</code> , or <code>until</code> loop.
<code>function</code>	Declares a function (alternative to <code>name() { ... }</code> syntax).
<code>select</code>	Begins a menu-based loop (useful for building simple menus).
<code>time</code>	Measures execution time of a command or block.
<code>then</code>	Used after <code>if</code> and <code>elif</code> conditions.
<code>break</code>	Exits from a loop prematurely.
<code>continue</code>	Skips the rest of the loop body and begins the next iteration.
<code>return</code>	Returns from a function with an optional status code.
<code>exit</code>	Exits the script with an optional status code.
<code>declare</code>	Declares variables and gives them attributes.
<code>local</code>	Declares variables local to a function.
<code>readonly</code>	Declares variables that cannot be modified.
<code>export</code>	Makes a variable available to child processes.
<code>let</code>	Performs arithmetic evaluation.
<code>[[and]]</code>	Used for advanced conditional expressions.
<code>!</code>	Logical NOT operator used in conditions.
<code>:</code>	No-op (does nothing; returns success).
<code>.</code>	Sources another script in the current shell.
<code>source</code>	Same as <code>.</code> — runs a script in the current shell.

Keyword	Description
trap	Defines actions to take when receiving signals.
eval	Evaluates and executes arguments as a Bash command.
test	Evaluates conditional expressions (alias of [...]).
true	Always returns success (0).
false	Always returns failure (1).



Some keywords like `[`, `]`, `!`, `:`, and `.` are not strictly "reserved words" in POSIX, but behave as such in Bash syntax.

A.12. Debugging Bash Scripts

Even the best-written Bash scripts can go wrong. When things don't behave as expected, debugging becomes essential. Bash provides several built-in tools and options to help you understand and fix issues in your scripts.

A.12.1. Enabling Debug Mode with `-x`

One of the most useful ways to debug a script is to enable **execution tracing**, which prints each command before it's executed.

You can do this in two ways:

Enable debugging for the entire script

```
#!/bin/bash
set -x # Turns on debug mode

echo "Start"
ls /tmp
set +x # Turns it off
echo "End"
```

Run the script with `-x`

```
bash -x myscript.sh
```

Each line and its arguments will be echoed to the terminal before being executed.



Use `set +x` to turn off debug mode selectively if you only want to trace part of the script.

A.12.2. Syntax Checking with **-n**

To check for syntax errors without executing the script:

```
bash -n myscript.sh
```

This is helpful for catching common mistakes like unmatched quotes or missing `fi`.

A.12.3. Verbose Mode with **-v**

Verbose mode prints each line of the script as it is **read**, not executed:

```
bash -v myscript.sh
```

This is useful to understand how Bash is parsing the script, especially when using dynamic evaluation.



You can also combine `-v` and `-x` for even more visibility:

```
bash -xv myscript.sh
```

A.12.4. Use **trap** to Catch Errors

You can catch errors and perform cleanup or log messages using the `trap` command:

```
#!/bin/bash
trap 'echo "Something went wrong at line $LINENO"; exit 1' ERR

echo "Doing something..."
cp source.txt destination/
```

This will print an error message with the line number if any command fails.

A.12.5. Print Debug Messages Manually

Sometimes, adding your own log messages is the easiest way to debug:

```
#!/bin/bash
debug() {
    echo "[DEBUG] $1"
}
```

```
debug "Starting script..."
myvar="hello"
debug "myvar is $myvar"
```



Avoid leaving debug print statements in production scripts unless they are guarded by a verbose/debug flag.

A.12.6. Use **return** and **exit** Codes

Always check and use proper exit statuses in your functions or scripts:

```
myfunc() {
    cp file1 file2 || return 1
}

myfunc || echo "Copy failed"
```

You can inspect the exit status of the last command using `$?`.

A.12.7. Debugging Loops and Conditionals

You can echo values inside loops and conditionals to see what's happening:

```
for file in *.txt; do
    echo "Checking $file"
    [[ -s "$file" ]] && echo "$file is not empty"
done
```

This technique is especially helpful when dealing with logic bugs.

A.12.8. Summary

Here's a quick overview of tools available for debugging Bash scripts:

- `set -x / bash -x` → Trace execution
- `set -v / bash -v` → Trace input
- `bash -n` → Syntax check
- `trap` → Error handling
- `echo / custom debug functions` → Manual inspection
- `$LINENO` → Shows the current line number
- `$?` → Shows last command exit code



When debugging complex scripts, isolate the problem by commenting out large sections and reintroducing them bit by bit.